

## **Tecnicatura Universitaria en Programación**

**Universidad Tecnológica Nacional**

### **Virtualización y Desarrollo en Linux**

**Arquitectura y Sistemas Operativos**

**Docente Titular**

Martín Aristiaran

**Docente Tutor**

Andrés Odiard

**Alumno:**

Fernando Picco (C10)

22 de octubre de 2025

## Índice

|                                                                                                      |    |
|------------------------------------------------------------------------------------------------------|----|
| Trabajo Integrador – Arquitectura y Sistemas Operativos.....                                         | 4  |
| 1. Introducción.....                                                                                 | 4  |
| 2. Marco Teórico .....                                                                               | 4  |
| Concepto de virtualización .....                                                                     | 4  |
| Beneficios de la virtualización .....                                                                | 4  |
| Hipervisores: Definición y Tipos .....                                                               | 5  |
| Imágenes ISO y snapshots .....                                                                       | 5  |
| Sistema Operativo Instalado (Ubuntu Server / Ubuntu Desktop) .....                                   | 6  |
| Python dentro de Linux .....                                                                         | 6  |
| Git y GitHub .....                                                                                   | 6  |
| 3. Caso Práctico: Gestión de Notas de Alumnos en Python dentro de una<br>Máquina Virtual Ubuntu..... | 7  |
| 4. Metodología Utilizada .....                                                                       | 10 |
| 5. Resultados Obtenidos.....                                                                         | 11 |
| 6. Conclusiones.....                                                                                 | 12 |
| 7. Bibliografía.....                                                                                 | 13 |
| 8. Anexos.....                                                                                       | 14 |
| Anexo 1 – Verificación de virtualización .....                                                       | 14 |
| Anexo 2 – Instalación de VirtualBox .....                                                            | 15 |
| Anexo 3 – Instalación de Ubuntu.....                                                                 | 15 |
| Anexo 4 – Instalación de VS Code.....                                                                | 30 |
| Anexo 5 – Configuración de Git .....                                                                 | 31 |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Anexo 6 – Creación del repositorio GitHub (desde la web) y primeros commits ..... | 34 |
| Anexo 7 – desarrollo del programa en la Máquina Virtual.....                      | 35 |

## Trabajo Integrador – Arquitectura y Sistemas Operativos

### 1. Introducción

La virtualización es una herramienta fundamental dentro del área de Arquitectura y Sistemas Operativos, ya que permite crear entornos aislados para pruebas, desarrollo y educación.

El presente trabajo tiene como objetivo aplicar los contenidos teóricos estudiados en la asignatura mediante la implementación práctica de una máquina virtual utilizando VirtualBox como hipervisor tipo 2, la instalación del sistema operativo Ubuntu en su interior y la posterior utilización del entorno para desarrollar un programa sencillo en Python.

Además, se integran conceptos de administración de sistemas, instalación de software, manejo de terminal Linux, control de versiones mediante Git y trabajo con repositorios remotos en GitHub.

El propósito del trabajo es demostrar la capacidad de configurar un entorno funcional desde cero y utilizarlo para resolver un problema real mediante programación.

### 2. Marco Teórico

#### ***Concepto de virtualización***

La virtualización consiste en crear un entorno informático simulado en lugar de uno físico. Esto incluye hardware, sistemas operativos y dispositivos de almacenamiento que normalmente existirían de forma independiente. Mediante la virtualización, un único equipo físico o servidor puede dividirse en múltiples máquinas virtuales auto-dependientes, que ejecutan sistemas operativos o aplicaciones diferentes mientras comparten los recursos del equipo anfitrión. Esta técnica mejora la escalabilidad y la utilización de recursos, al mismo tiempo que ayuda a reducir costos de infraestructura, consumo energético y mantenimiento (Microsoft Azure, s.f.).

#### ***Beneficios de la virtualización***

La virtualización aporta múltiples beneficios clave para las organizaciones informáticas. En primer lugar, permite reducir gastos de TI al consolidar servidores físicos en máquinas virtuales, aprovechando mejor el hardware. Además, mejora la continuidad de negocio

al reducir los tiempos de inactividad y facilitar la recuperación ante desastres. Asimismo, aumenta la eficiencia y productividad del equipo de TI al simplificar tareas de mantenimiento y despliegue. Otro beneficio es ofrecer mayor autonomía a los equipos de desarrollo (DevOps), al permitir crear entornos aislados rápidamente sin afectar producción. Finalmente, la virtualización favorece la sostenibilidad ambiental, ya que al disminuir el número de servidores físicos se consume menos energía y se reduce la huella de carbono (IBM, s.f.).

### ***Hipervisores: Definición y Tipos***

Según IBM (s. f.), un hipervisor es la capa de software que se ubica entre el hardware físico de un equipo y las máquinas virtuales (VM) que se crean en él. Su función es coordinar los recursos físicos (CPU, memoria, almacenamiento) de modo que múltiples máquinas virtuales puedan ejecutarse simultáneamente sin interferir entre ellas. Dentro de este contexto, se identifican dos tipos principales:

- ✓ Tipo 1 (bare-metal o nativo): El hipervisor se ejecuta directamente sobre el hardware físico, sin sistema operativo anfitrión. Es común en entornos de servidor, donde la eficiencia y el rendimiento son críticos.
- ✓ Tipo 2 (hospedado): El hipervisor corre como una aplicación dentro de un sistema operativo existente (anfitrión). Este modelo es habitual en ordenadores de escritorio o entornos de pruebas, aunque implica una sobrecarga de rendimiento porque depende del sistema operativo anfitrión.

La adopción de hipervisores permite, por ejemplo, consolidar múltiples servidores virtuales sobre un solo hardware físico, mejorar la utilización de recursos, facilitar la migración y replicación de sistemas, así como aislar entornos. En tu trabajo, el uso de VirtualBox corresponde a un hipervisor de tipo 2, ya que se instaló dentro de un sistema operativo anfitrión (Windows).

### ***Imágenes ISO y snapshots***

Una *imagen ISO* es un archivo que guarda una “copia sector-por-sector” de un medio óptico (CD, DVD o Blu-ray), incluyendo el sistema de archivos, los datos y la estructura original. Se utiliza frecuentemente para distribuir sistemas operativos o programas,

porque al montarse o grabarse imita exactamente el medio físico original (Comunidad académica Enciclopedia, s.f.).

Los *snapshots* o instantáneas son capturas del estado de un sistema, volumen o máquina virtual en un momento preciso. No se trata de una copia completa, sino de un “fotograma” que registra los cambios a partir de ese punto, permitiendo restaurar el sistema a ese estado rápidamente, con menor consumo de almacenamiento y sin detener su operación (AO Data Cloud, 2023).

### ***Sistema Operativo Instalado (Ubuntu Server / Ubuntu Desktop)***

La edición Desktop de Ubuntu está diseñada para el uso diario en computadoras personales y portátiles: incluye interfaz gráfica, aplicaciones de usuario y entorno visual completo. Por otro lado, la edición Server está optimizada para servidores: no incluye GUI de forma predeterminada, se administra principalmente por la línea de comandos y está pensada para alojar servicios, bases de datos o procesos de producción. Mientras que Desktop prioriza la usabilidad y experiencia visual, Server prioriza el rendimiento, eficiencia de recursos y estabilidad para tareas de servidor (LinuxOpsys, 2023).

### ***Python dentro de Linux***

La mayoría de las distribuciones Linux, incluido Ubuntu, incorporan Python instalado por defecto porque se utiliza para herramientas del sistema, automatización y scripts administrativos. Linux ofrece un entorno muy cómodo para programar en Python, ya que permite ejecutar archivos directamente desde la terminal mediante comandos como `python3 archivo.py`.

Para este trabajo práctico se utilizó Python dentro de la máquina virtual Ubuntu, de modo que el programa pudiera desarrollarse, probarse y ejecutarse en un entorno real de Linux, utilizando herramientas como VS Code, la terminal y Git. Este enfoque permitió comprobar la integración entre virtualización y programación.

### ***Git y GitHub***

Git es un sistema de control de versiones diseñado para registrar y gestionar los cambios realizados en los archivos a través del tiempo. Permite dividir el trabajo en ramas, fusionar modificaciones y mantener un historial detallado de un proyecto.

GitHub, por su parte, es una plataforma basada en la nube donde se alojan repositorios creados con Git. Ofrece almacenamiento, colaboración, gestión de cambios, revisión por otros usuarios y sincronización entre el repositorio local y remoto.

La combinación de ambos (Git y GitHub) permite trabajar de forma eficiente tanto localmente como en equipo: los desarrolladores hacen cambios en su máquina, los registran con Git y luego los comparten en GitHub, donde pueden revisarlos, integrarlos o colaborar ( GitHub, s.f.).

### **3. Caso Práctico: Gestión de Notas de Alumnos en Python dentro de una Máquina Virtual Ubuntu**

#### Introducción

Como parte del Trabajo Práctico Integrador, se solicitó elaborar un caso práctico utilizando la máquina virtual creada durante el desarrollo del proyecto. El objetivo principal es demostrar la instalación, configuración y uso de un entorno de desarrollo dentro del sistema operativo Ubuntu, ejecutado en VirtualBox, y resolver un problema real mediante un programa en Python.

Para este caso práctico, se desarrolló un programa llamado *“notas\_alumnos.py”*, cuyo propósito es gestionar las notas de estudiantes, calcular promedios individuales y brindar resultados generales del curso. Este ejemplo fue elegido porque integra conceptos básicos de programación (secuenciales, condicionales, repetitivas y listas) y permite validar que el entorno virtual funciona correctamente con Python y Git.

#### Descripción del problema

El docente solicita un programa que permita:

1. Ingresar datos de varios alumnos, incluyendo:
  - ✓ Nombre del estudiante
  - ✓ Tres notas numéricas
2. Calcular automáticamente el promedio de cada uno.
3. Determinar si el alumno está aprobado o desaprobado, considerando:

- ✓ Aprobación si el promedio es mayor o igual a 7.
- 4. Registrar múltiples alumnos, utilizando un ciclo repetitivo.
- 5. Finalmente, mostrar los resultados generales del curso:
  - ✓ Cantidad total de alumnos,
  - ✓ Cantidad de aprobados y desaprobados,
  - ✓ Promedio general de la cursada,
  - ✓ Mejor promedio obtenido.

El programa debía funcionar desde la terminal de Ubuntu, dentro de la VM creada previamente.

#### Desarrollo de la solución

El script en Python fue desarrollado en Visual Studio Code dentro de Ubuntu y versionado paso a paso con Git.

Funciones y estructuras aplicadas:

- ✓ Entradas de datos (input)
- ✓ Conversión de tipos (float)
- ✓ Estructuras condicionales (if/else)
- ✓ Estructura repetitiva (while)
- ✓ Manejo básico de listas para almacenar nombres y promedios
- ✓ Cálculos matemáticos (promedios)
- ✓ Formato de salida utilizando f-strings

El programa permite repetir la carga de alumnos hasta que el usuario elija finalizar.

#### Pruebas realizadas

En la terminal de Ubuntu se ejecutó el programa con:

```
python3 notas_alumnos.py
```

Se probaron diferentes combinaciones de notas y múltiples alumnos para verificar:

- ✓ Correctos cálculos de promedio



- ✓ Funcionamiento del condicional de aprobación
- ✓ Acumulación de resultados generales
- ✓ Salida formateada en pantalla

Las pruebas mostraron resultados correctos, entre ellos:

- ✓ Alumnos aprobados y desaprobados
- ✓ Promedio general
- ✓ Mejor promedio
- ✓ Presentación ordenada de todos los datos

### Control de versiones (Git y GitHub)

El desarrollo del caso práctico se versionó desde la máquina virtual. Se realizaron múltiples commits, entre ellos:

- ✓ "Prueba 1: prom y est" (primer cálculo de promedio simple)
- ✓ "Prueba 2: carga + alumn + estd grales" (versión final con repetición y estadísticas)
- ✓ "Prueba 3: carga varios alumn., estad. grales y posib. repetir" (versión final con repetición, estadísticas y posibilidad de repetir)

Se utilizó:

- ✓ `git add notas_alumnos.py`
- ✓ `git commit -m "mensaje descriptivo"`
- ✓ `git push` (con token de GitHub almacenado correctamente)

El repositorio remoto utilizado: [https://github.com/FernandoPicco/TPI\\_AySO.git](https://github.com/FernandoPicco/TPI_AySO.git)

### Conclusión del caso práctico

El desarrollo del programa permitió validar que:

- ✓ La máquina virtual Ubuntu se encuentra correctamente configurada,
- ✓ Es posible instalar aplicaciones de desarrollo (VS Code, Python),
- ✓ El entorno responde adecuadamente a la ejecución de programas,
- ✓ Git está funcional para versionar código dentro del sistema invitado,

- ✓ Se integran los contenidos de Programación I y AySO.

El caso práctico resuelve un problema real de manera clara y representa adecuadamente el uso del entorno virtual como herramienta educativa.

#### 4. Metodología Utilizada

Esta sección describe los pasos concretos realizados durante el proyecto, tal como indica la plantilla :

- 1- Instalación de VirtualBox y verificación de virtualización
  - ✓ Se descargó VirtualBox desde la web oficial.
  - ✓ Se validó que la CPU tuviera activada la tecnología de virtualización desde el Administrador de Tareas (Ver capturas en Anexos).
2. Descarga de la imagen ISO de Ubuntu
  - ✓ Se descargó inicialmente Ubuntu 24.04.3, pero debido a un error típico al iniciar el modo live, se reemplazó por Ubuntu 22.04.5 LTS.
3. Creación y configuración de la máquina virtual
  - ✓ Asignación de memoria RAM, CPU y disco.
  - ✓ Modificación de memoria de video para mejorar rendimiento (Ver capturas en Anexos).
4. Instalación del sistema operativo Ubuntu
  - ✓ Proceso guiado con la ISO (Ver capturas en Anexos).
5. Instalación de herramientas necesarias
  - ✓ Visual Studio Code.
  - ✓ Python (verificado).
  - ✓ Git.
6. Configuración de GitHub y autenticación mediante token personal
  - ✓ Configuración del user.name y user.email.
  - ✓ Generación y almacenamiento del token para evitar reingresos repetidos.
7. Creación del repositorio local y remoto
  - ✓ Carpeta *TPI\_AySO*.
  - ✓ *git init*
  - ✓ Creación del repositorio vacío en GitHub.

- ✓ Conexión con *git remote add origin*.

#### 8. Desarrollo del programa en Python

- ✓ Creación del archivo *notas\_alumnos.py*.
- ✓ Pruebas incrementales.
- ✓ Realización de commits significativos.

### 5. Resultados Obtenidos

Los resultados obtenidos durante el desarrollo del Trabajo Práctico Integrador fueron positivos, ya que se logró completar la instalación, configuración y uso de una máquina virtual Ubuntu dentro de VirtualBox, así como la creación y ejecución de un programa en Python versionado con Git y sincronizado con GitHub. A continuación, se detallan los principales logros y también las dificultades encontradas durante el proceso:

#### Resultados alcanzados

- ✓ Instalación exitosa de VirtualBox como hipervisor tipo 2.
- ✓ Configuración de la máquina virtual y correcta asignación de recursos (RAM, CPU, almacenamiento y memoria de video).
- ✓ Instalación funcional de Ubuntu 22.04.5 LTS dentro de la máquina virtual.
- ✓ Instalación y funcionamiento correcto de Python, Git y Visual Studio Code dentro del entorno Linux.
- ✓ Desarrollo del programa “*notas\_alumnos.py*”, incluyendo carga repetitiva de alumnos, cálculo de promedios y estadísticas generales.
- ✓ Ejecución correcta del programa desde la terminal sin errores.
- ✓ Configuración completa del control de versiones con Git.
- ✓ Creación del repositorio remoto en GitHub y sincronización mediante commits y push.
- ✓ Historial de commits correcto reflejando las distintas etapas del desarrollo.

#### Dificultades presentadas y cómo fueron resueltas

##### 1. Error al instalar Ubuntu 24.04.3 LTS en VirtualBox

Durante la instalación inicial, la versión 24.04 presentó un error habitual al intentar iniciar el entorno live dentro de VirtualBox. Este problema suele deberse a

incompatibilidades temporales entre la versión de Ubuntu y el hipervisor.

*Solución:* se reemplazó por la versión LTS 22.04.5, la cual se instaló correctamente sin errores.

## 2. Problemas al vincular Git con GitHub mediante token personal

El uso de token generó varias dificultades:

- ✓ El token no se aceptaba al momento del push.
- ✓ Hubo que generar más de un token debido a errores en los permisos o al no copiarlo correctamente.
- ✓ GitHub Desktop no funcionaba dentro de la máquina virtual y fue necesario trabajar 100% desde terminal.

***Solución:***

- ✓ Se generó un nuevo token con los permisos correctos (scope repo).
- ✓ Se configuró Git con user.name y user.email.
- ✓ Se almacenaron las credenciales para evitar reingresarlas.
- ✓ Luego de estos pasos, el push funcionó correctamente.

## 3. Problemas menores durante la edición del código en VS Code

Hubo dificultades iniciales con el teclado (símbolos como ~, {}, #), comandos y ejecución del programa.

***Solución:***

- ✓ Utilizar las teclas AltGr + 3 (#) que en Windows usualmente la indica Ctrl+Alt+3.

## 6. Conclusiones

El desarrollo del Trabajo Práctico Integrador permitió integrar de manera efectiva los contenidos vistos en la asignatura Arquitectura y Sistemas Operativos, aplicando virtualización, administración de entornos Linux y programación con Python dentro de un entorno controlado. La creación de la máquina virtual y la instalación de Ubuntu proporcionaron una experiencia concreta sobre cómo funcionan los sistemas operativos invitados y cuáles son los desafíos habituales de compatibilidad.

Uno de los aspectos más significativos del trabajo fue la configuración de Git y su vinculación con GitHub. Este proceso resultó particularmente desafiante debido al uso de tokens personales de acceso, que generaron errores y obligaron a repetir el procedimiento varias veces hasta lograr una configuración estable y funcional. Esta dificultad permitió comprender la importancia de la autenticación segura y de los permisos adecuados en entornos profesionales de desarrollo.

Asimismo, la primera instalación fallida de Ubuntu 24.04 dentro de VirtualBox mostró que no todas las versiones del sistema operativo funcionan correctamente en entornos virtualizados, y que la elección de una versión LTS más estable (22.04) puede resolver problemas de compatibilidad.

Finalmente, el caso práctico en Python demostró que el entorno virtual configurado funciona correctamente, permitiendo desarrollar, ejecutar y versionar código sin inconvenientes. El trabajo permitió afianzar conocimientos tanto de programación estructurada como de administración de sistemas en entornos virtualizados, y representa una experiencia formativa completa y aplicable al ámbito profesional.

## 7. Bibliografía

GitHub. (s.f.). *Acerca de GitHub y Git*. Obtenido de <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>

AO Data Cloud. (27 de noviembre de 2023). *Snapshots: Definición y ventajas clave*. Obtenido de <https://aodatacloud.es/blog/snapshots-definicion-y-ventajas/>

Comunidad académica Enciclopedia. (s.f.). *Imagen ISO*. Obtenido de <https://encyclopedia.pub/entry/33349>

datos.gob.es. (20 de octubre de 2021). *¿Qué es un diccionario de datos y por qué es importante?*. Blog de datos.gob.es. Obtenido de <https://datos.gob.es/es/blog/que-es-un-diccionario-de-datos-y-por-que-es-importante>

IBM. (s.f.). *¿What is virtualization?* Obtenido de <https://www.ibm.com/mx-es/think/topics/virtualization>

LinuxOpsys. (25 de mayo de 2023). *Ubuntu Desktop vs Ubuntu Server: Diferencias clave y casos de uso*. Obtenido de <https://linuxopsys.com/ubuntu-desktop-vs-ubuntu-server-differences>

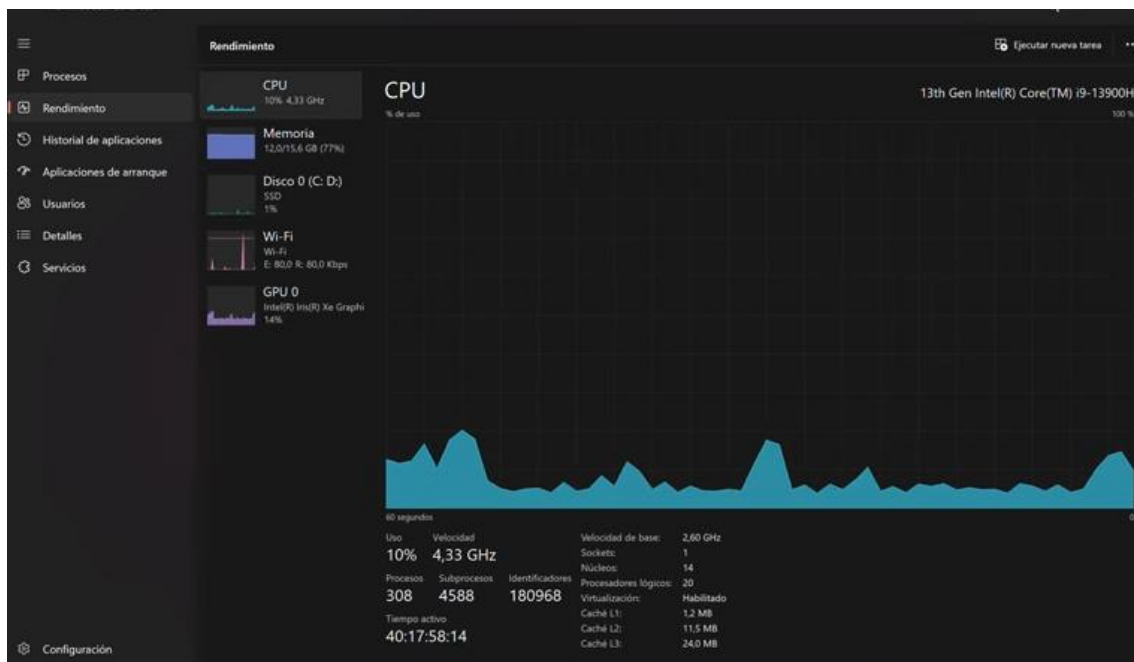
Microsoft Azure. (s.f.). *¿Qué es la virtualización?* Obtenido de <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-virtualization>

SAP Spain Centro de Noticias. (2 de agosto de 2023). *La importancia del análisis de datos en una empresa*. Obtenido de <https://news.sap.com/spain/2023/08/la-importancia-del-analisis-de-datos-en-una-empresa/>

## 8. Anexos

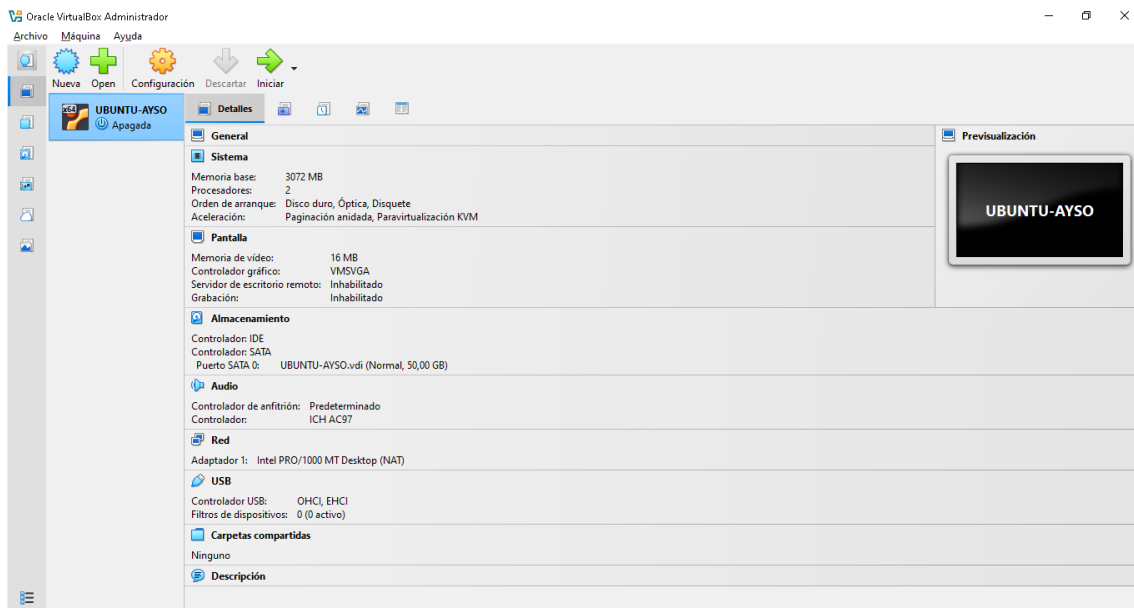
### Anexo 1 – Verificación de virtualización

Verifiqué mi CPU esté habilitada para ejecutar la virtualización. Para eso fui al “Administrador de tareas” en la parte de rendimiento encontré que la misma estaba habilitada.



## Anexo 2 – Instalación de VirtualBox

Instalación de Virtual Box desde su página oficial (para Windows y luego procedí a descargar el paquete de extensiones de Virtual Box)

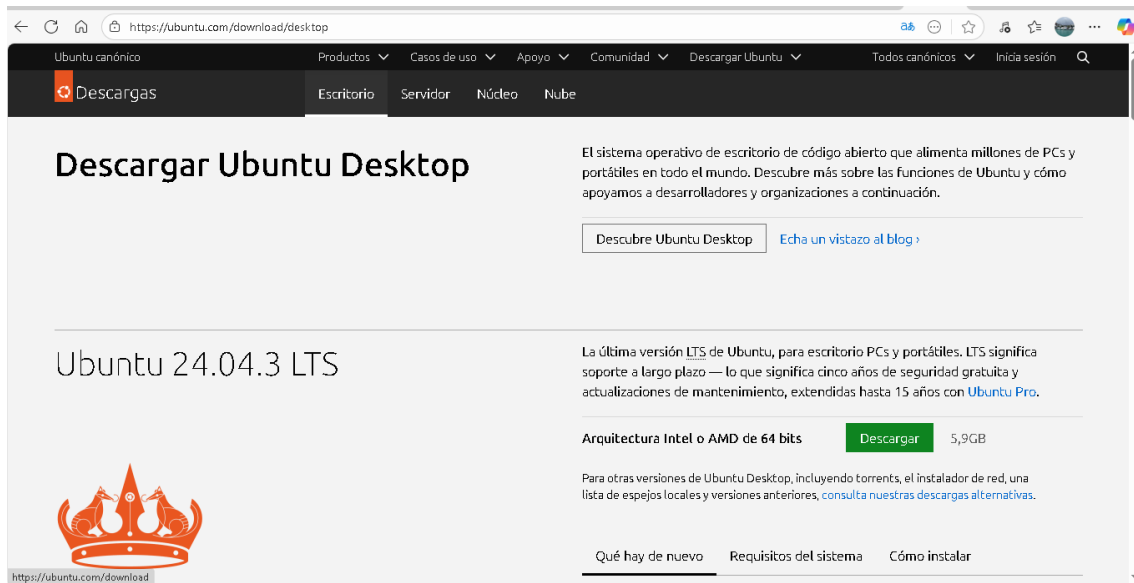


## Anexo 3 – Instalación de Ubuntu

1- Desde la página web de Ubuntu descargué la última versión 24.04.3 LTS.

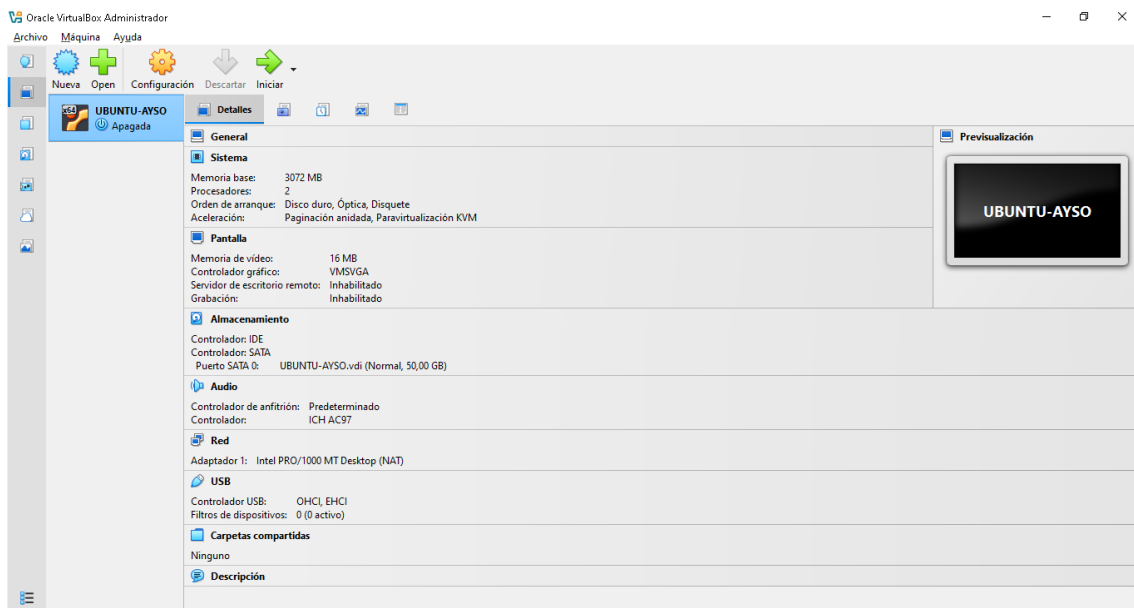
# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos



## 2- Creación de la máquina virtual para realizar el TPI

- Utilicé imagen ISO de Ubuntu Server

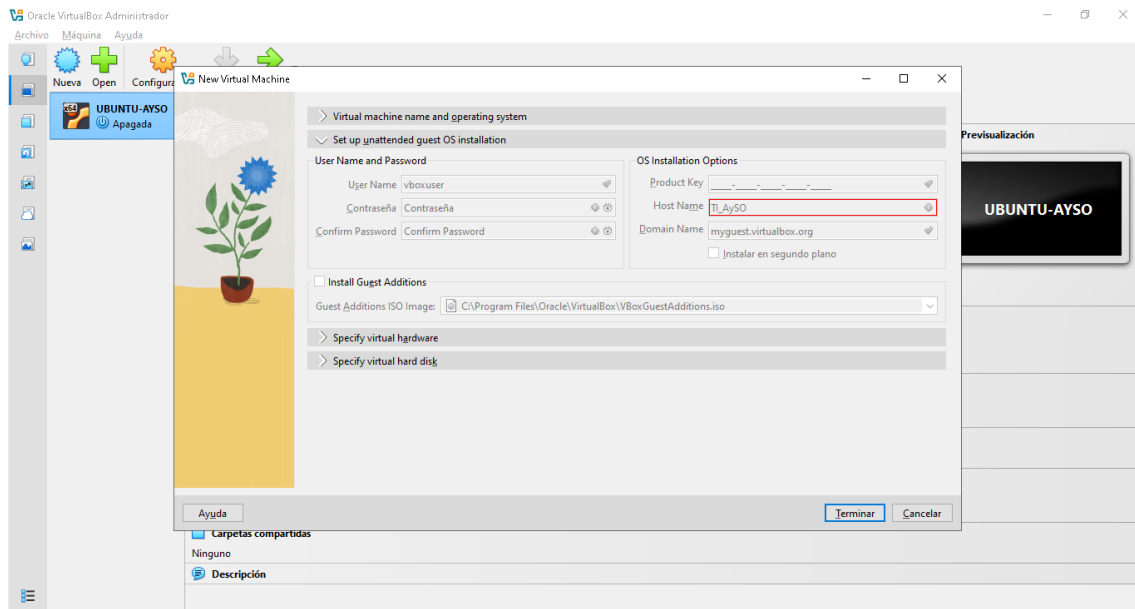
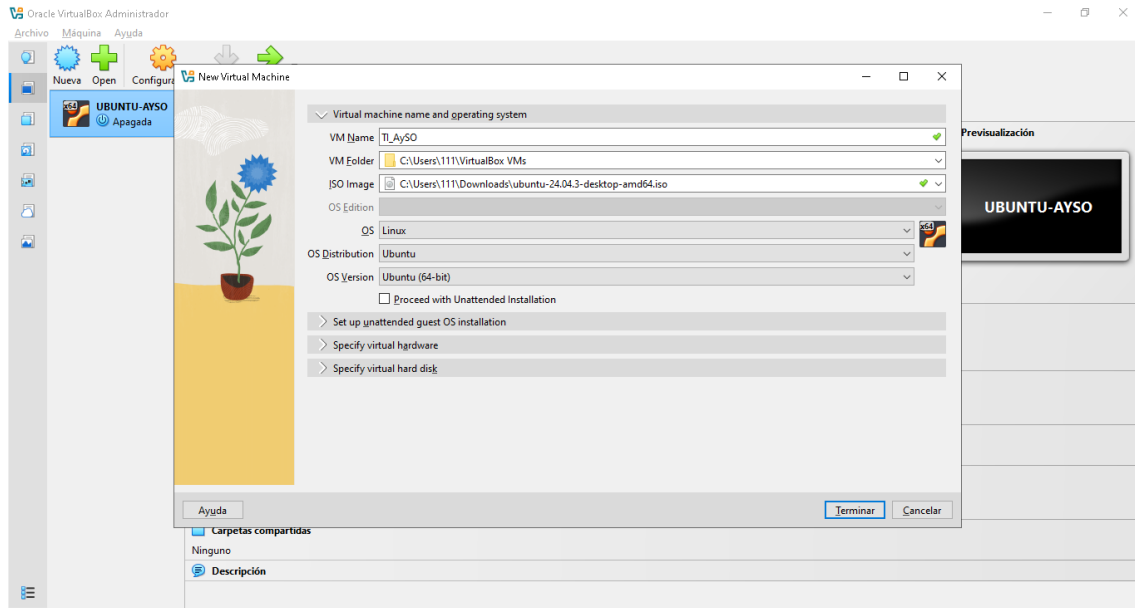


- Se definió nombre de usuario



# TRABAJO PRÁCTICO INTEGRADOR

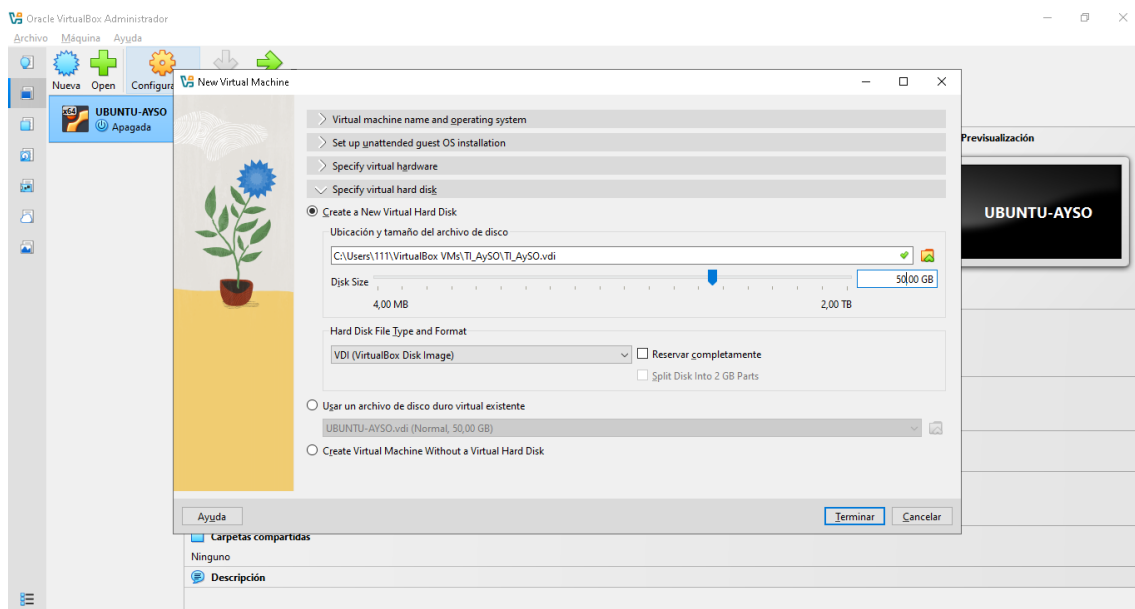
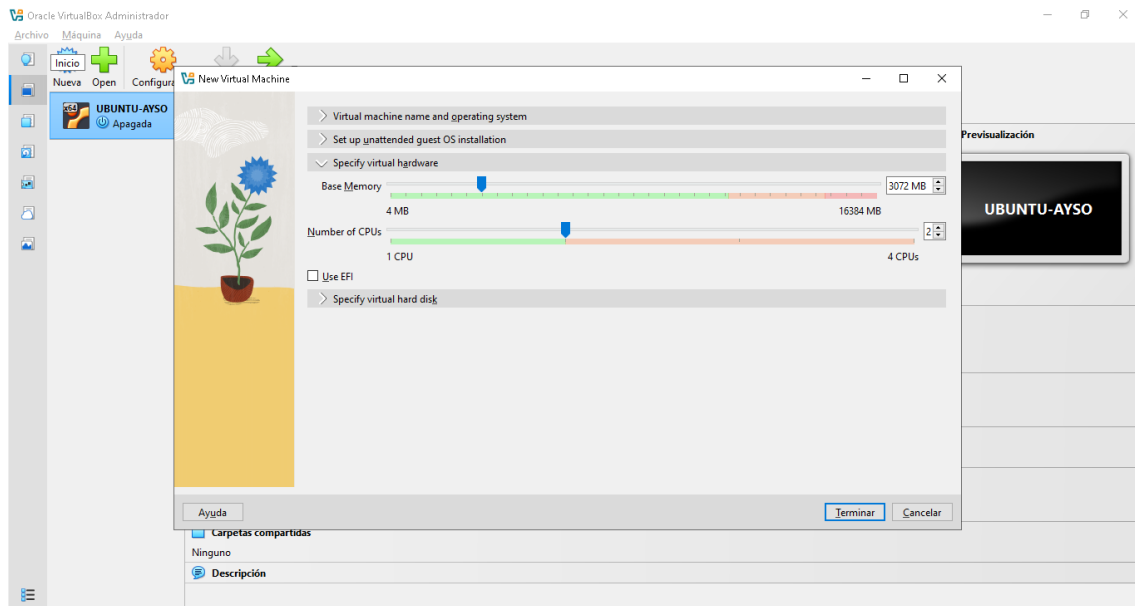
## Arquitectura y Sistemas Operativos



- Asignación de recursos (memoria, almacenamiento y núcleos)

# TRABAJO PRÁCTICO INTEGRADOR

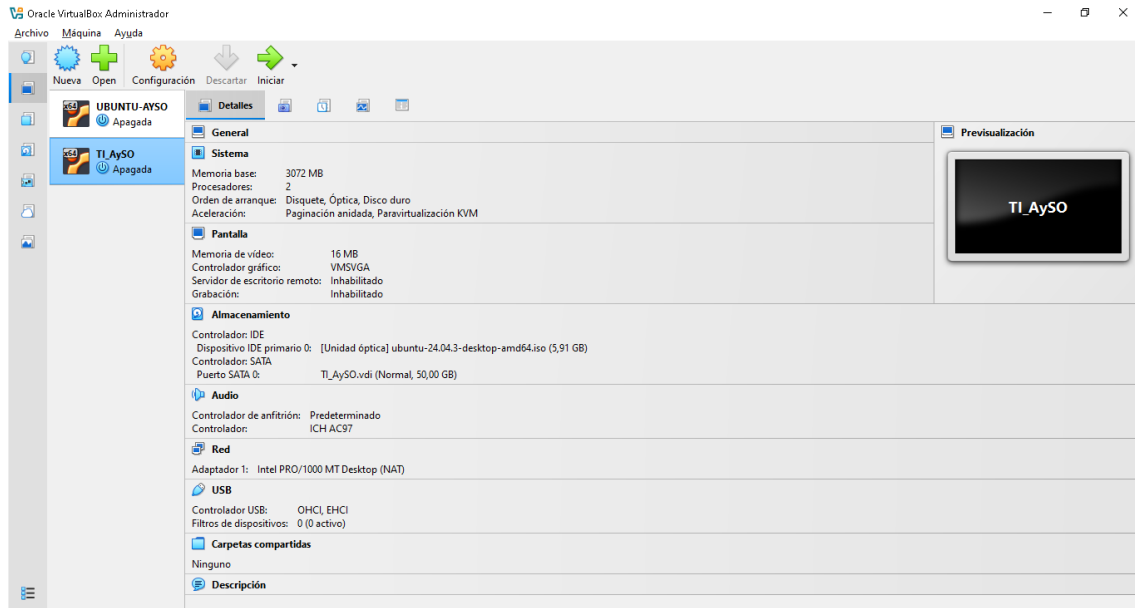
## Arquitectura y Sistemas Operativos



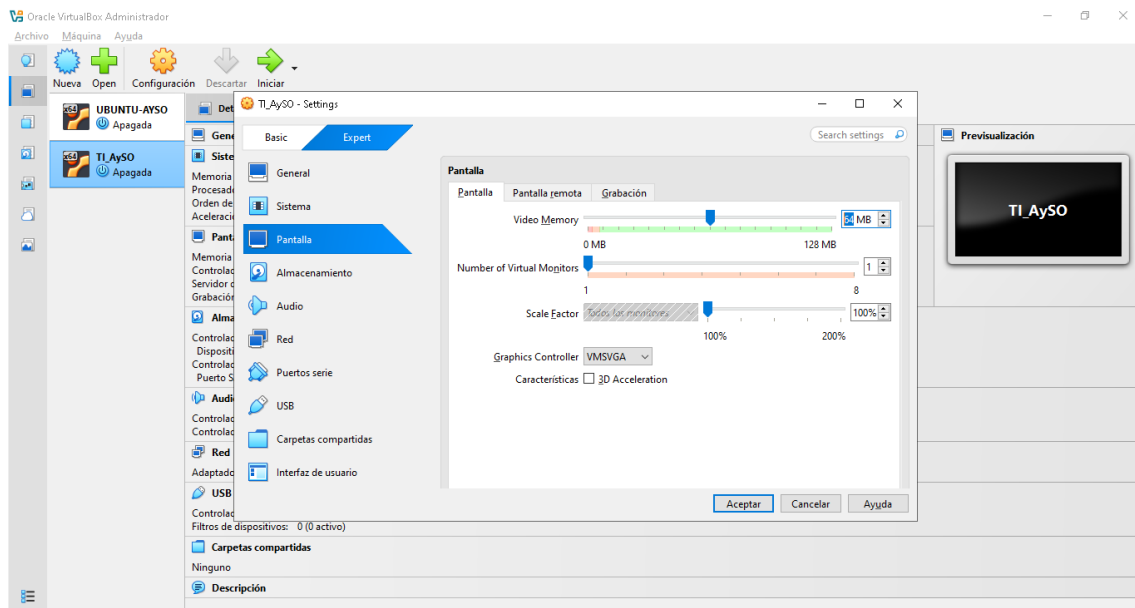
- Máquina virtual configurada: "TI\_AySO"

## TRABAJO PRÁCTICO INTEGRADOR

### Arquitectura y Sistemas Operativos



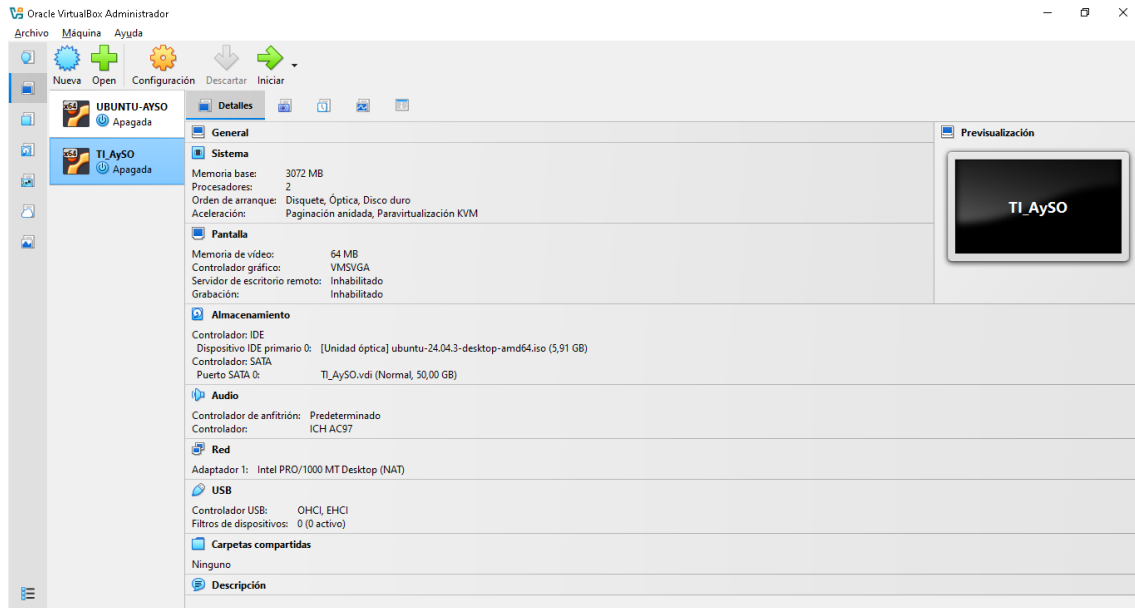
- Modificación (Aumentar la memoria de video): permite que Ubuntu funcione fluido.



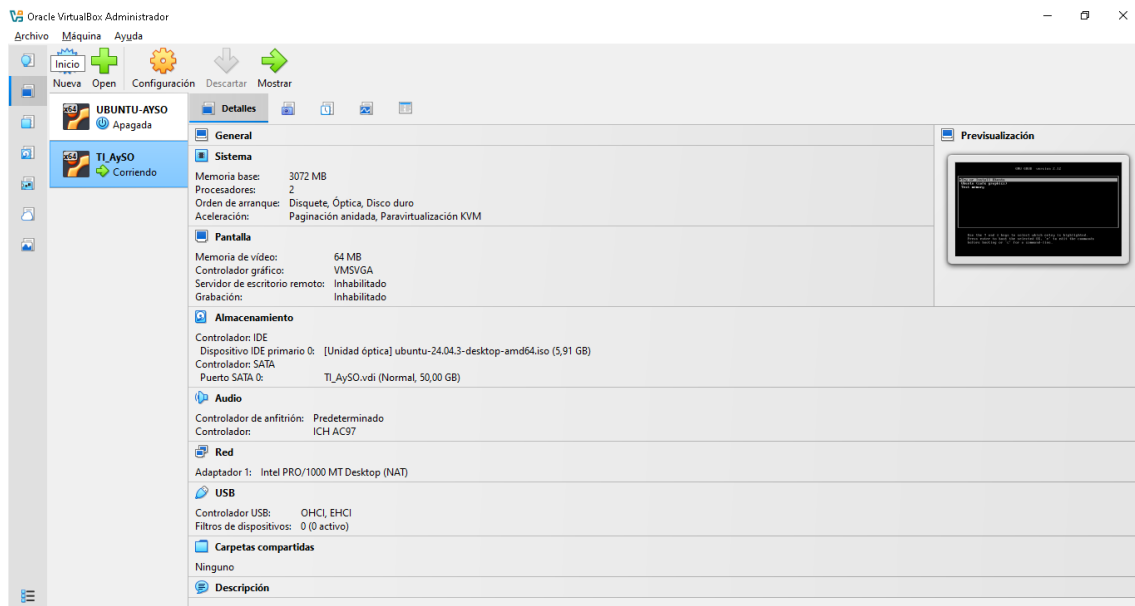
- Máquina configurada (versión final)

# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos



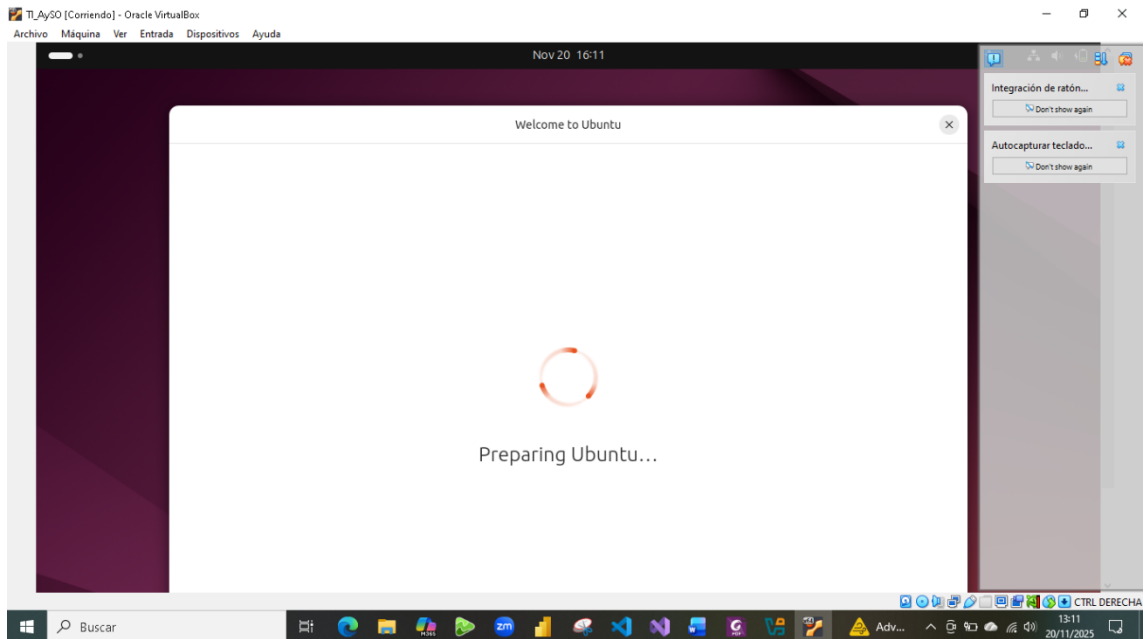
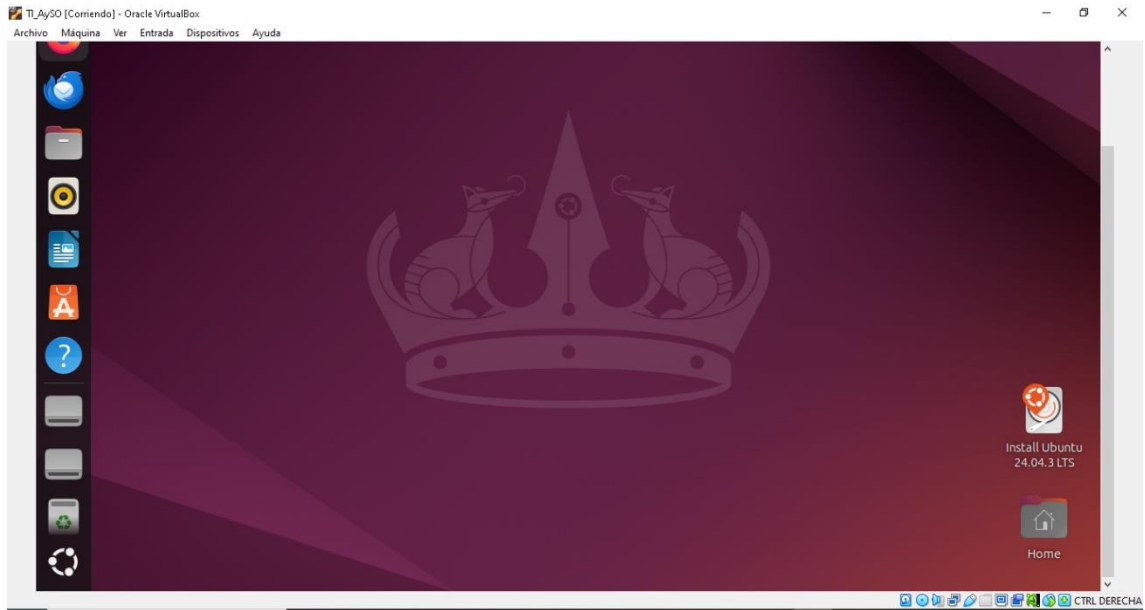
### - Inicialización de la máquina virtual



### - Instalar Ubuntu desde el escritorio del “modo live” de Ubuntu

# TRABAJO PRÁCTICO INTEGRADOR

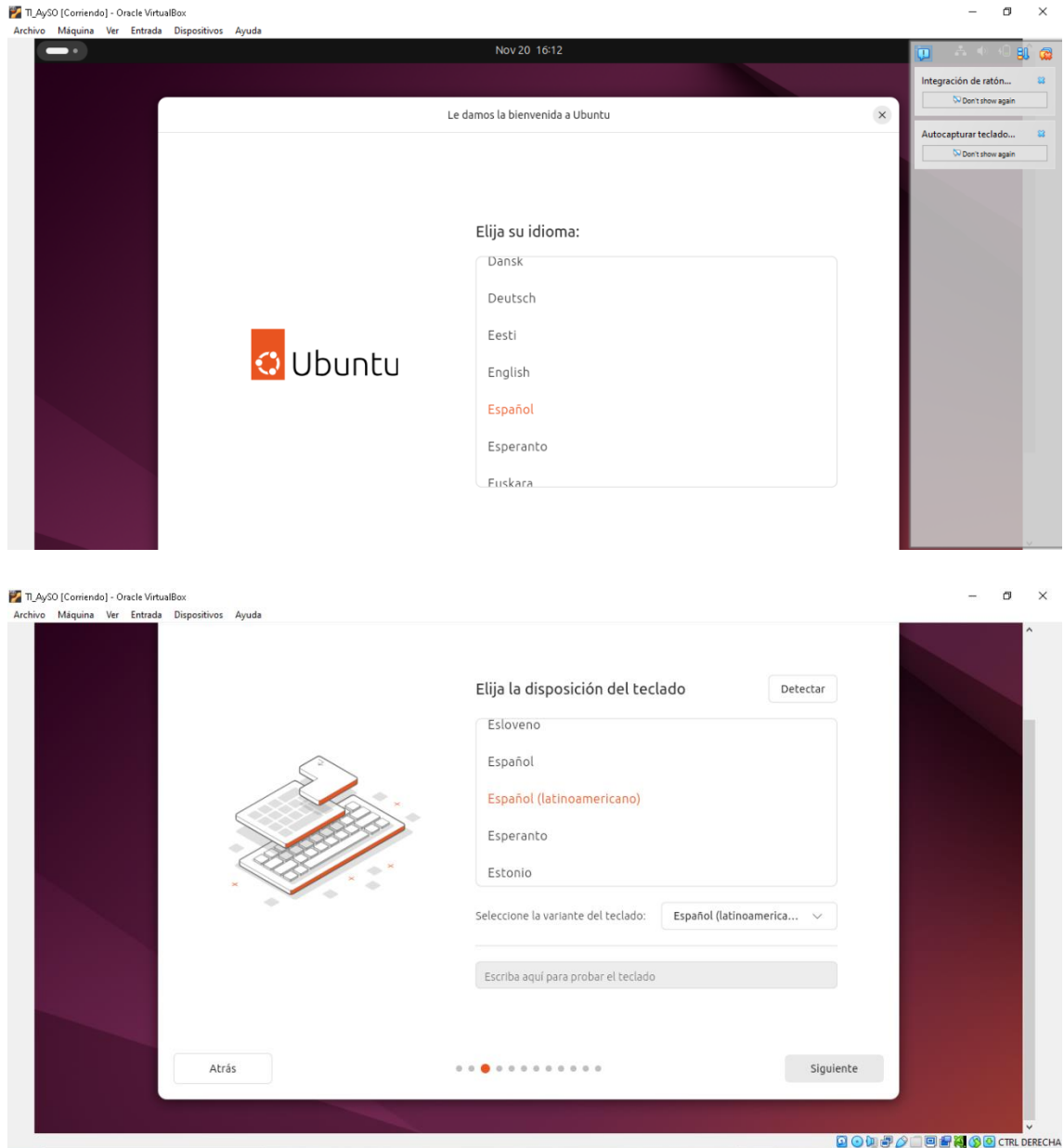
## Arquitectura y Sistemas Operativos



- Configuración

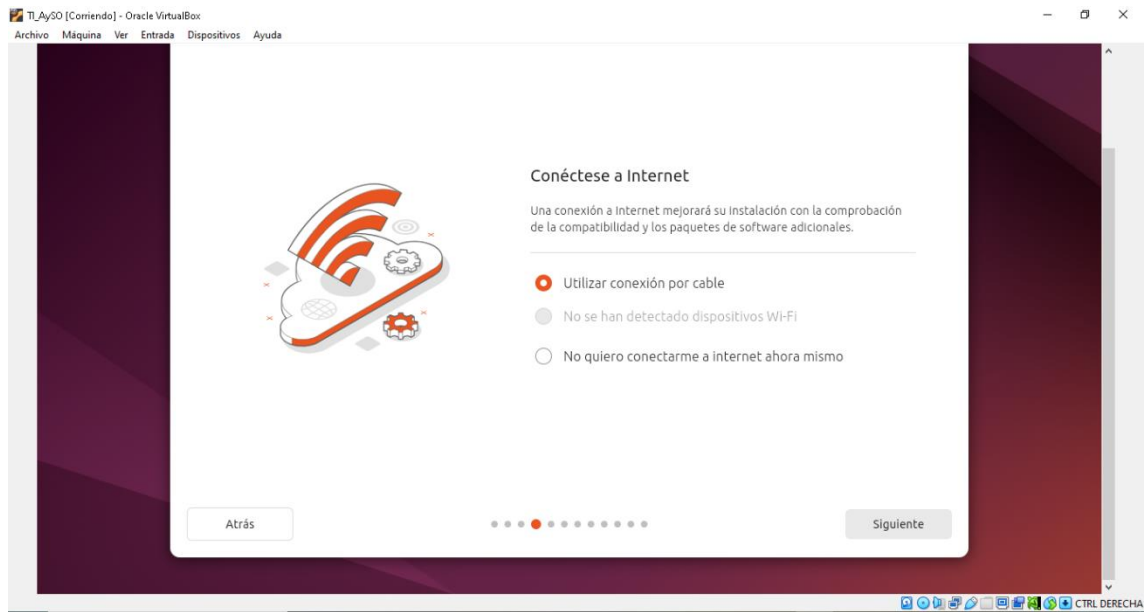
# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos

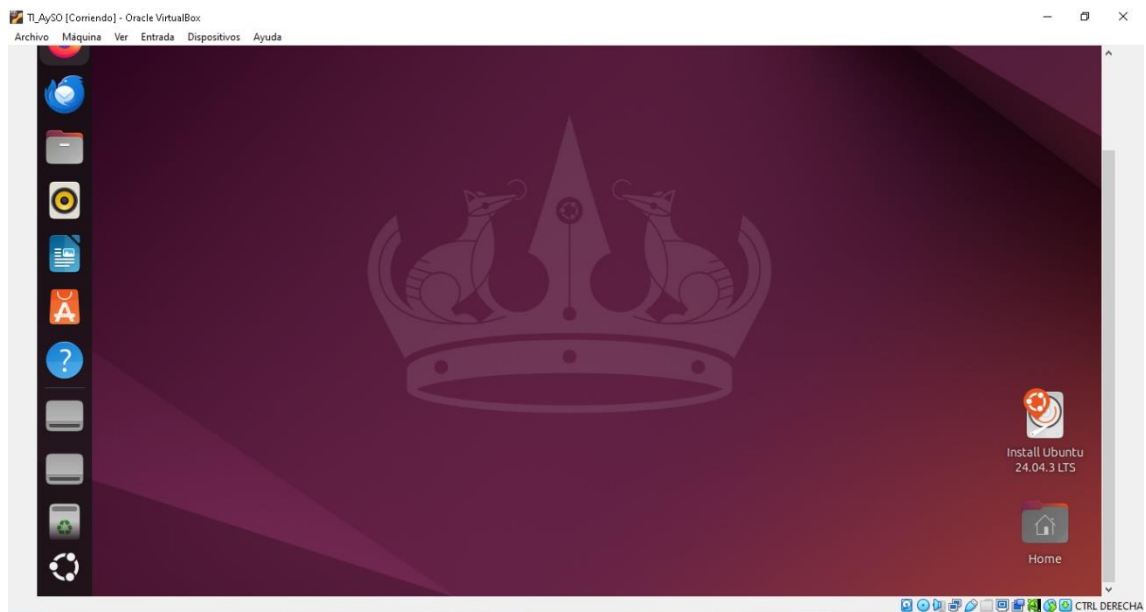


## TRABAJO PRÁCTICO INTEGRADOR

### Arquitectura y Sistemas Operativos

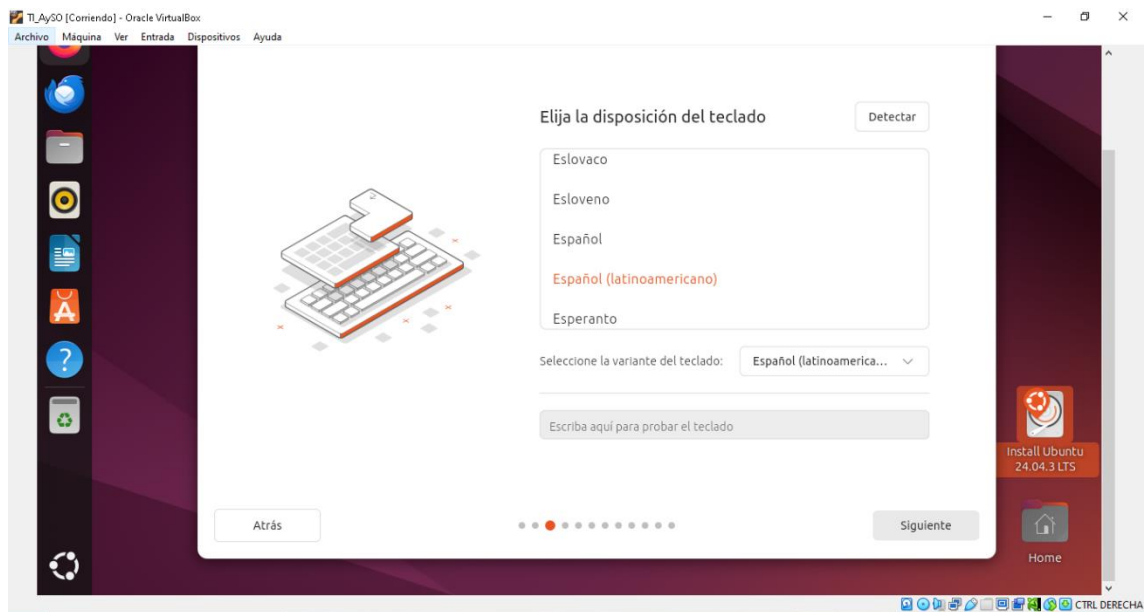
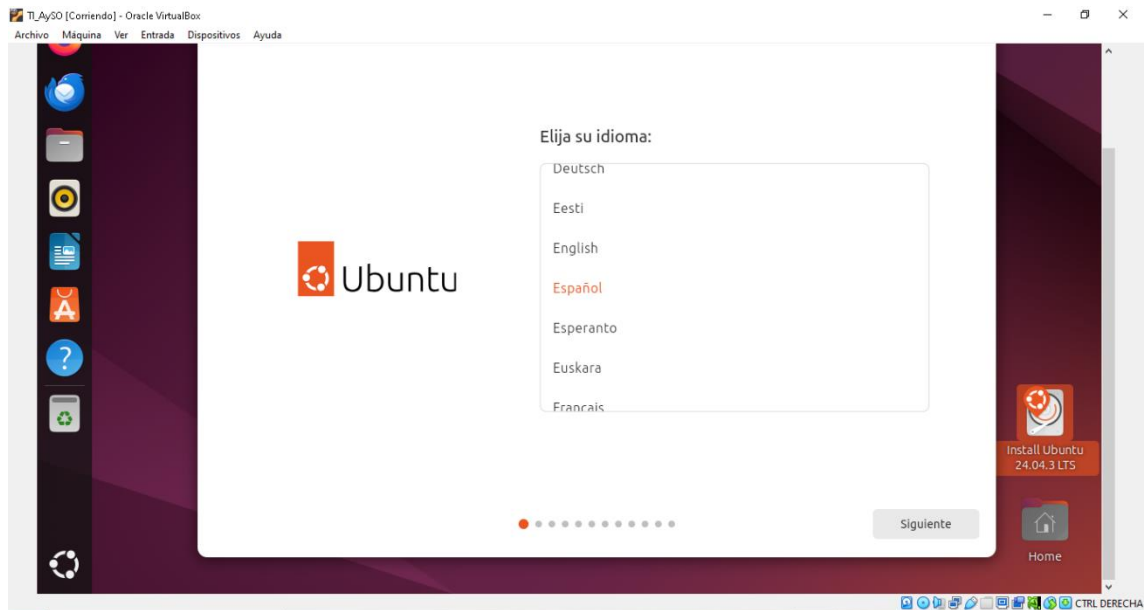


#### - Instalación de Ubuntu 24.04.3 LTS manualmente



# TRABAJO PRÁCTICO INTEGRADOR

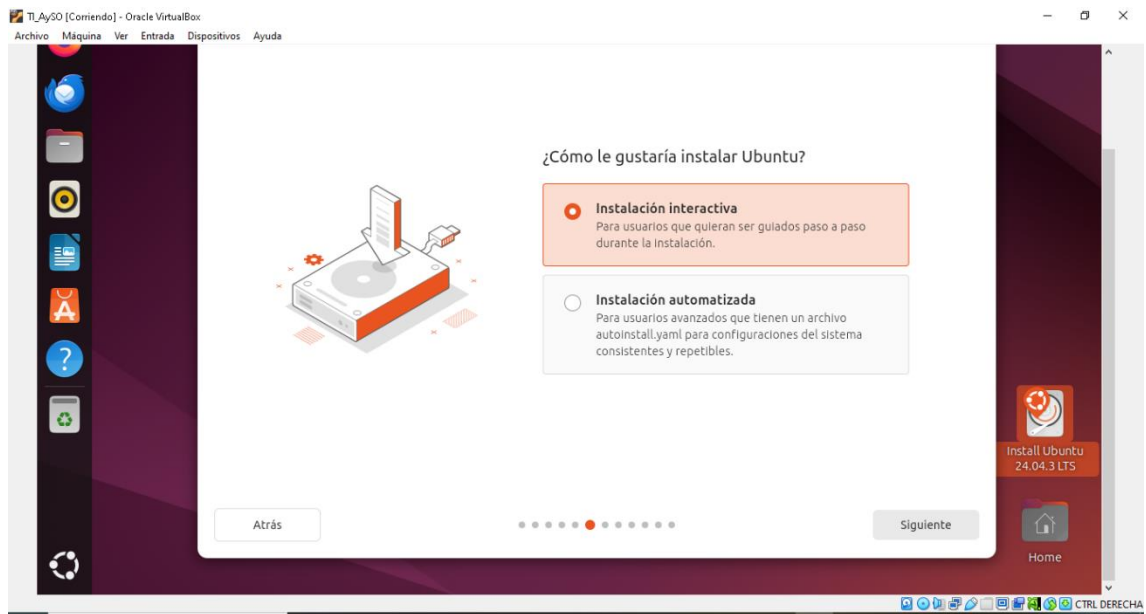
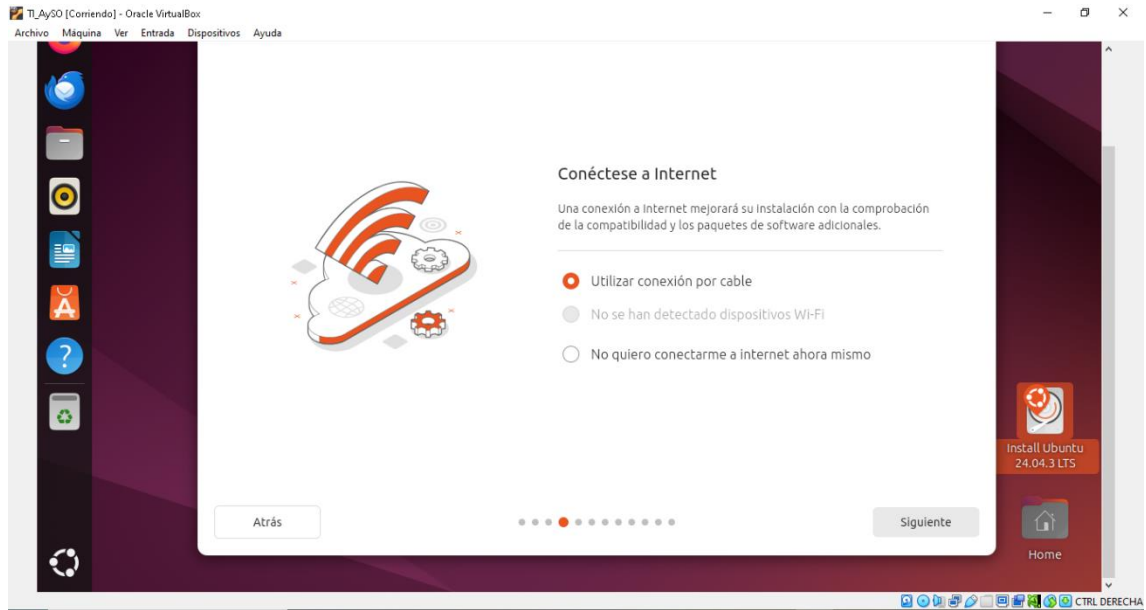
## Arquitectura y Sistemas Operativos





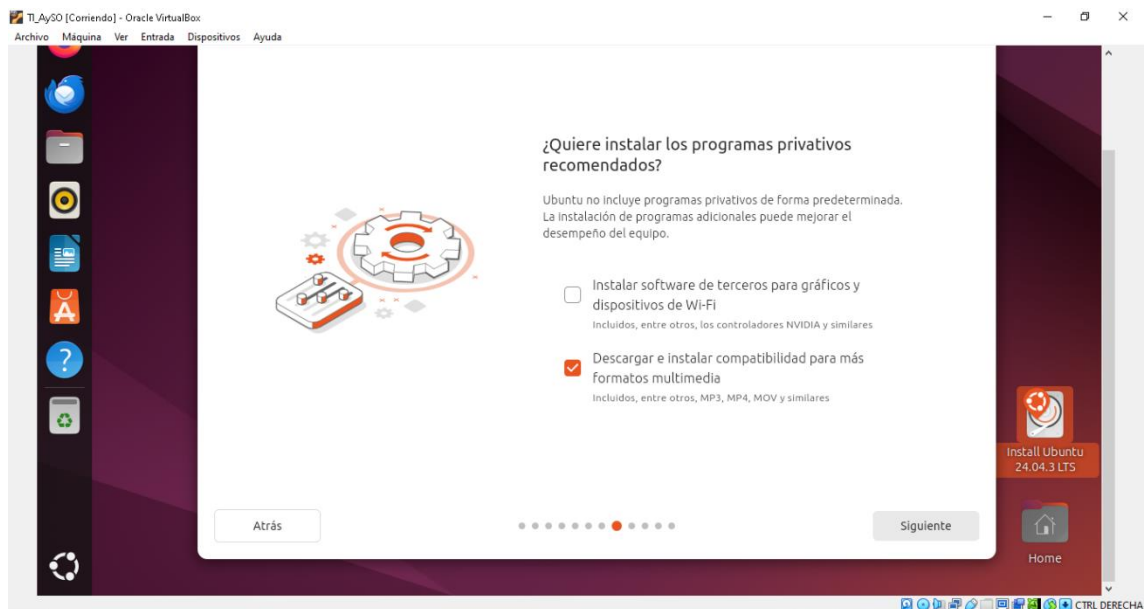
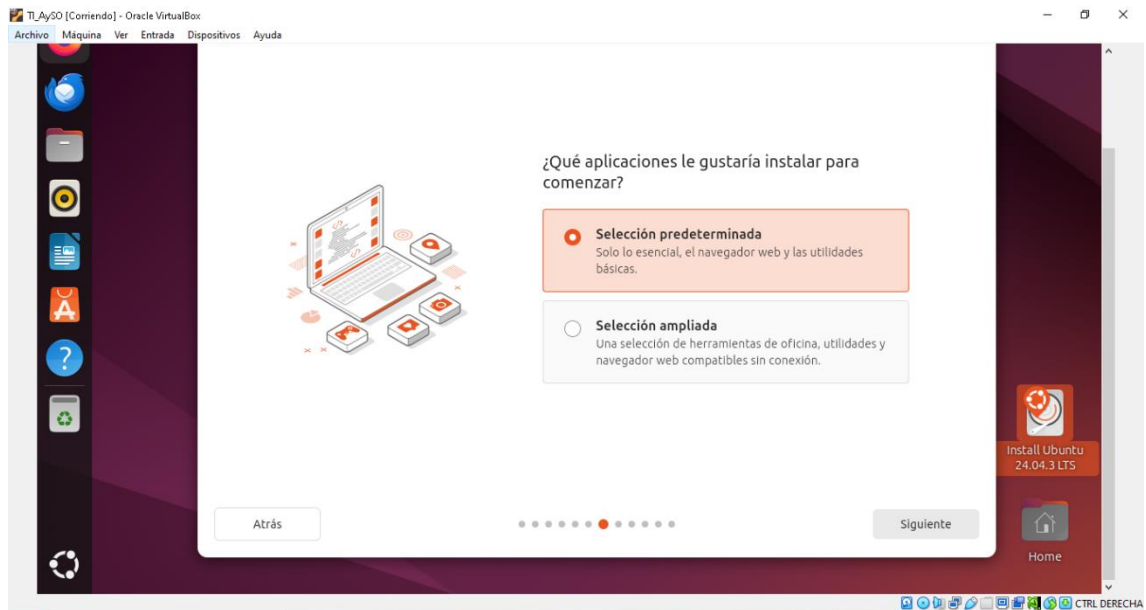
# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos



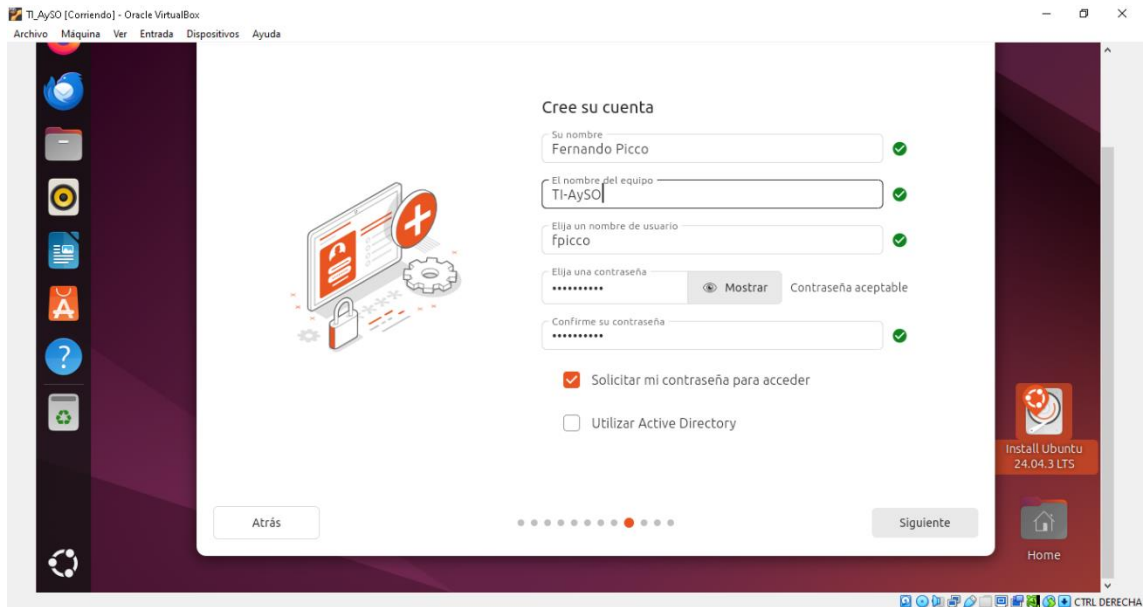
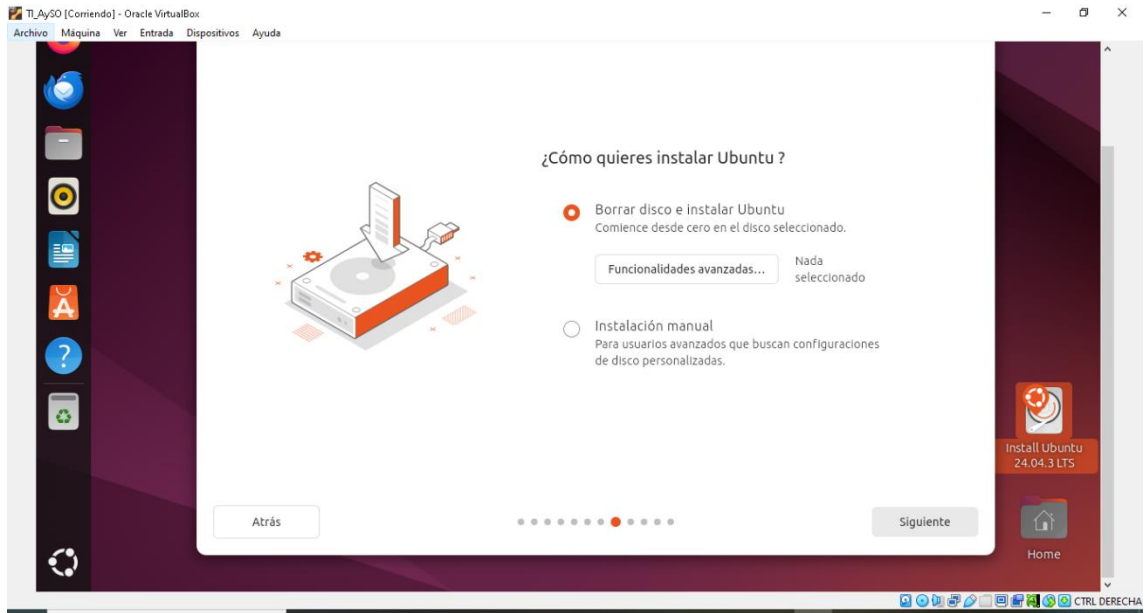
# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos

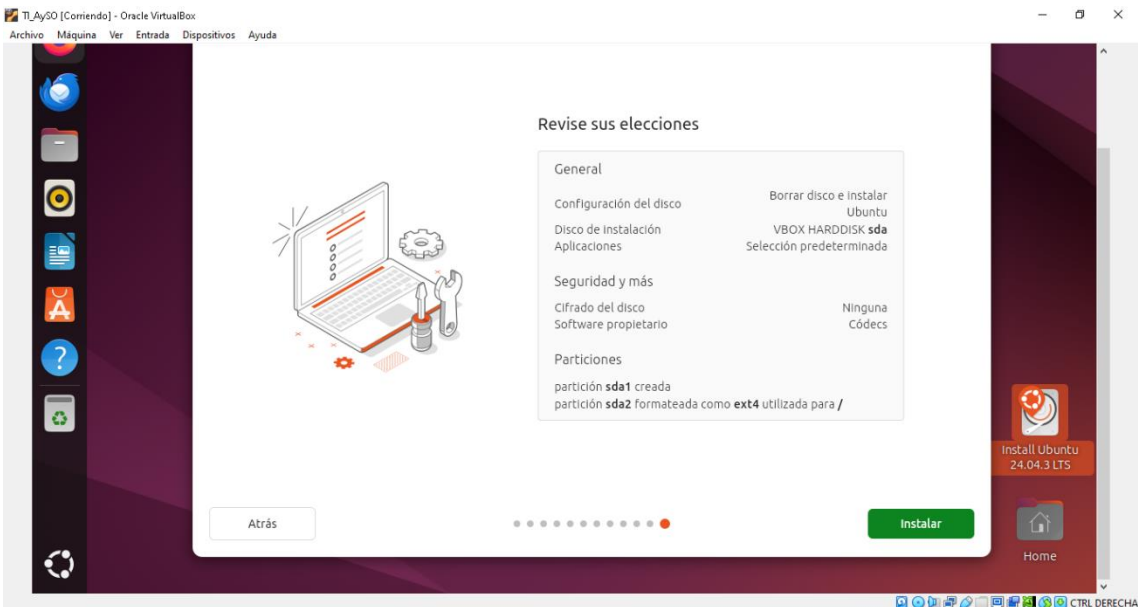
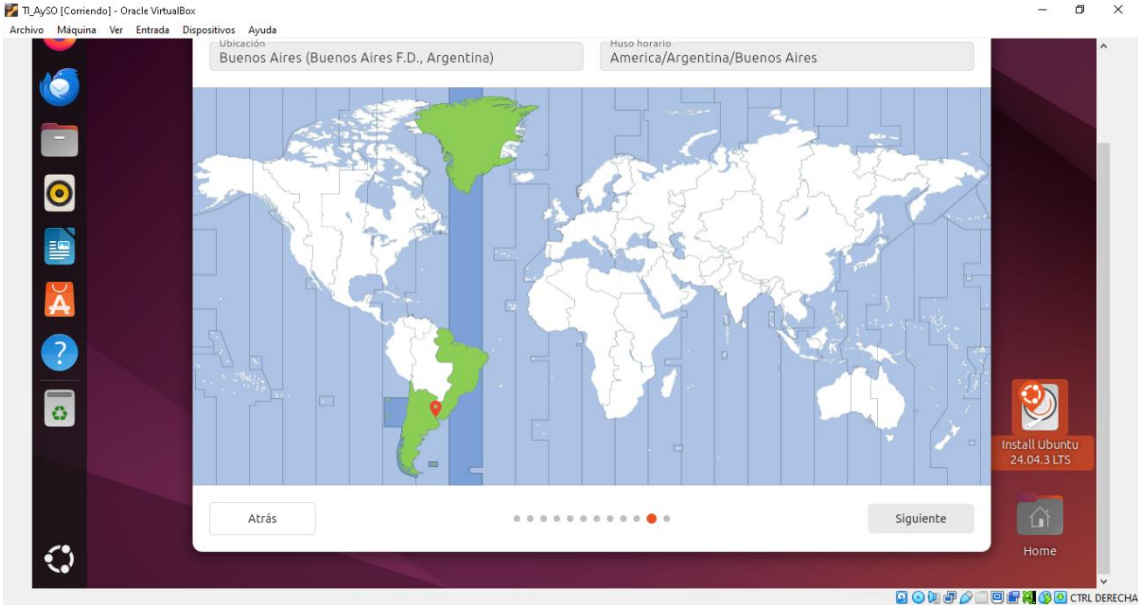


# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos

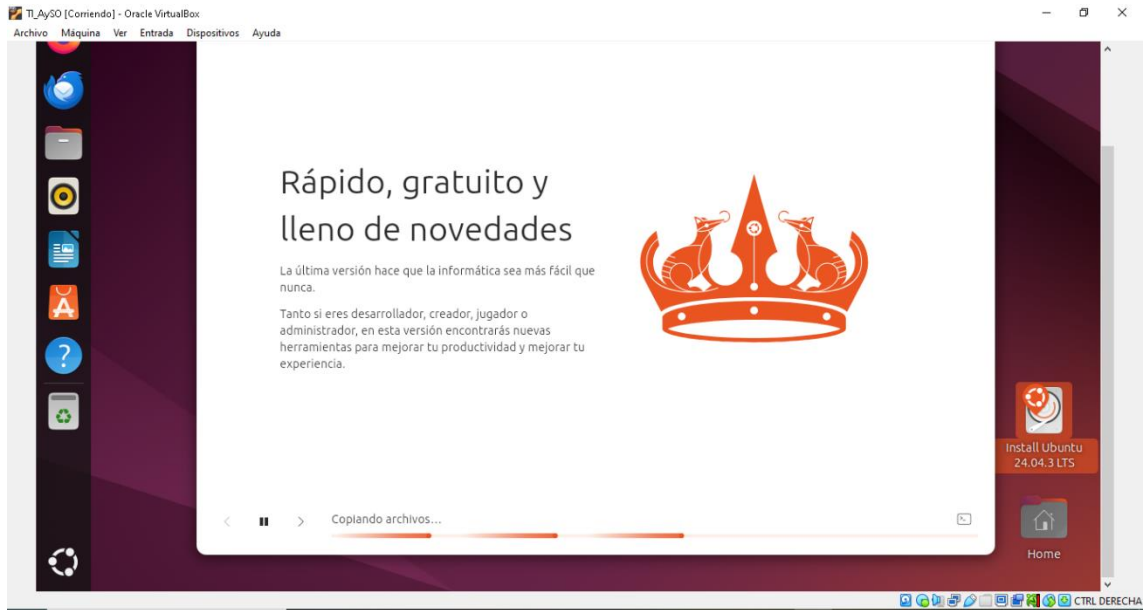


## Arquitectura y Sistemas Operativos

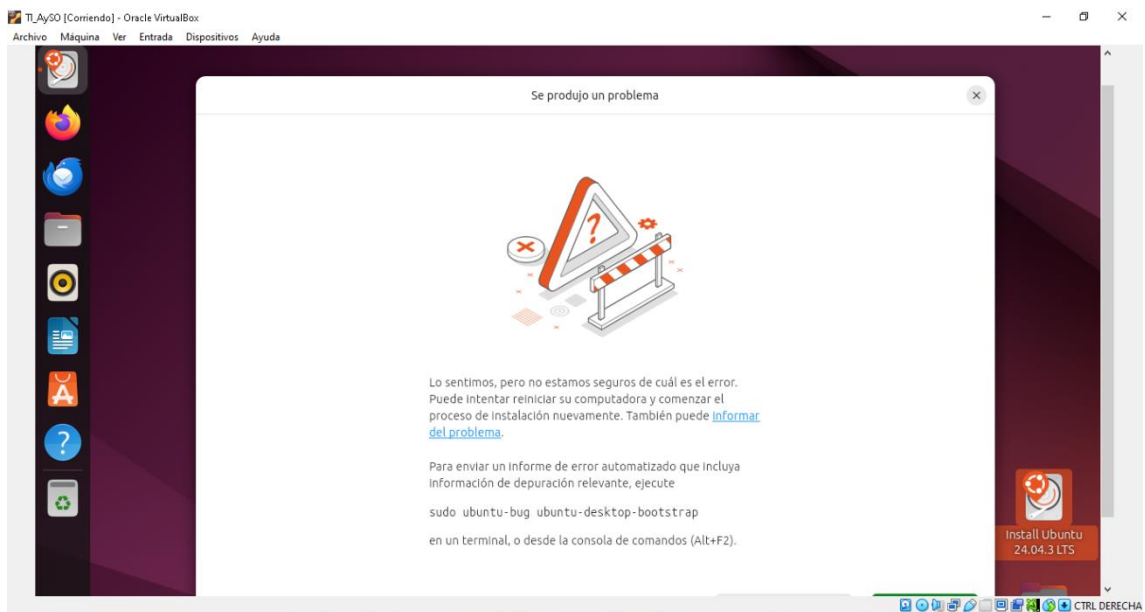


## TRABAJO PRÁCTICO INTEGRADOR

### Arquitectura y Sistemas Operativos



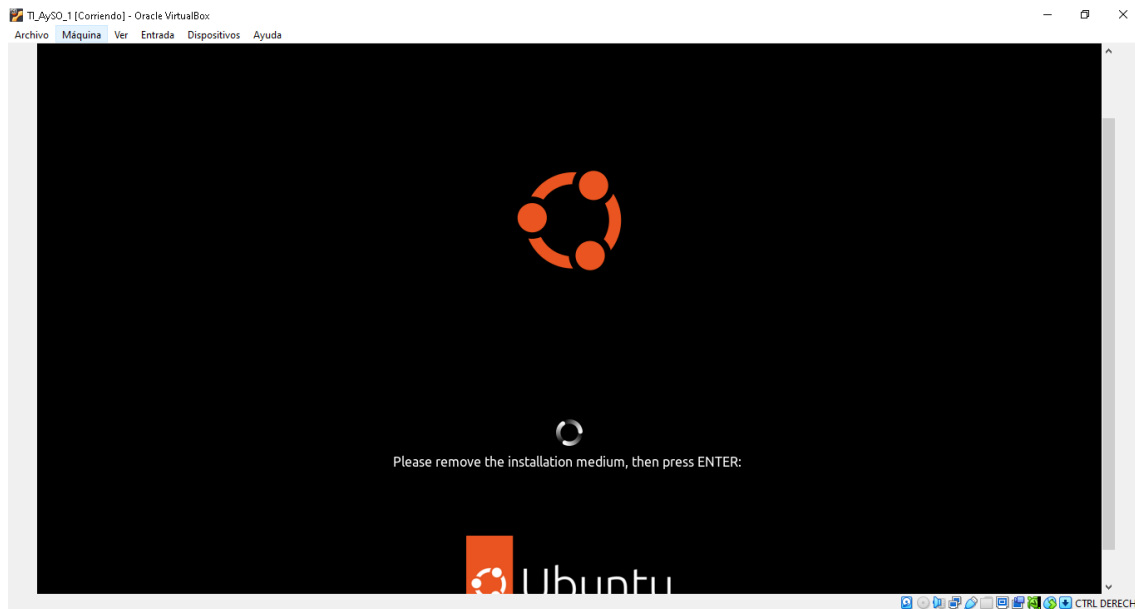
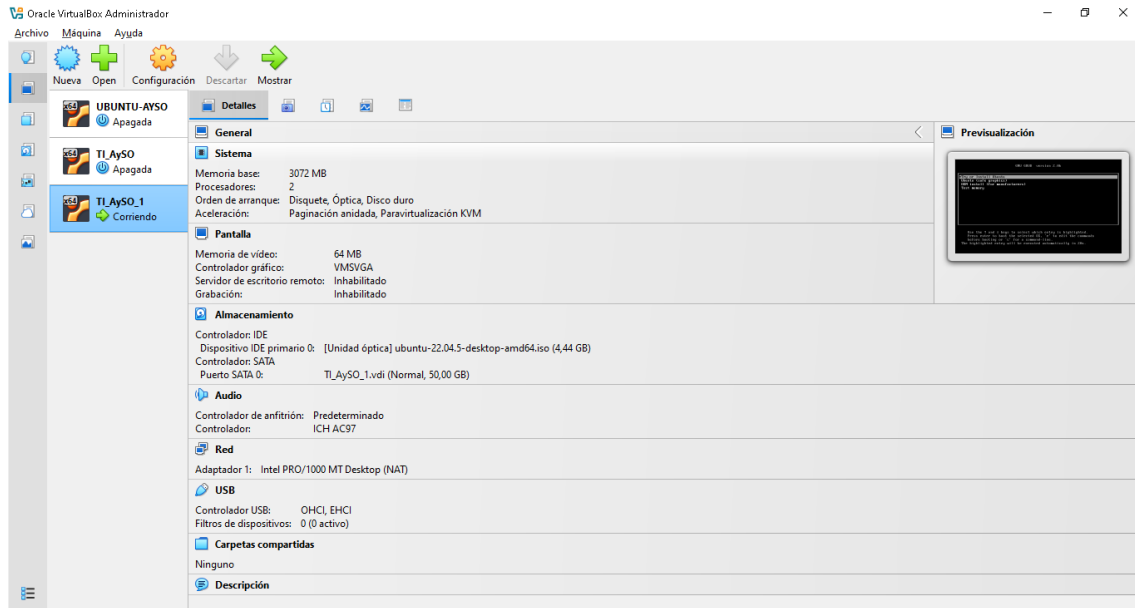
- Según ChatGPT es el error típico de Ubuntu 24.04 en VirtualBox cuando hay un problema con la compatibilidad del sistema “live”.



- Procedí a descargar la versión Ubuntu 22.04.5 LTS

## TRABAJO PRÁCTICO INTEGRADOR

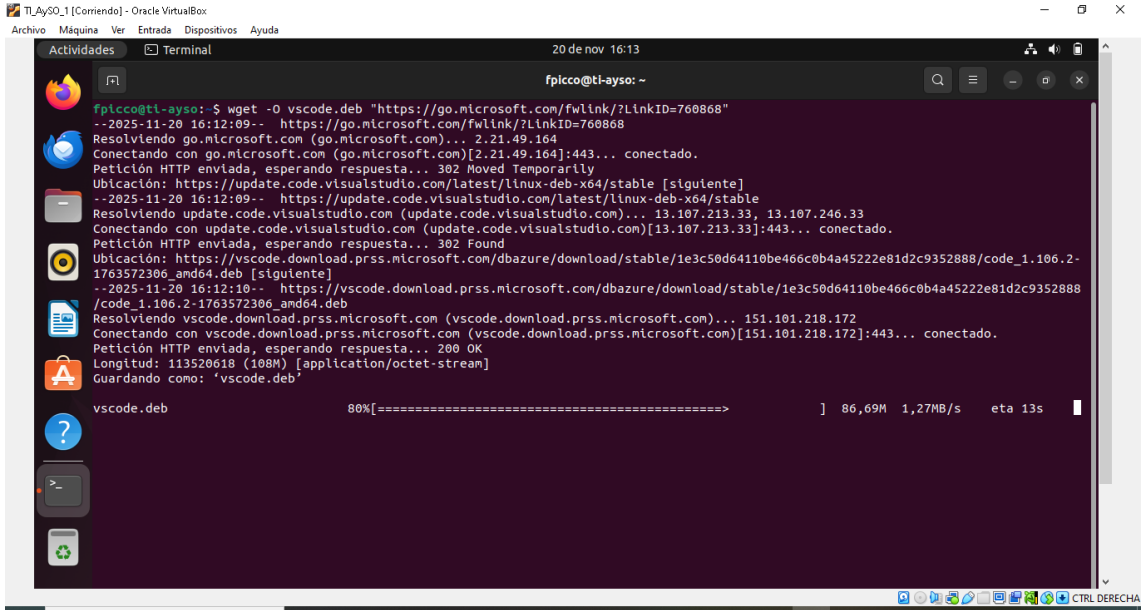
### Arquitectura y Sistemas Operativos



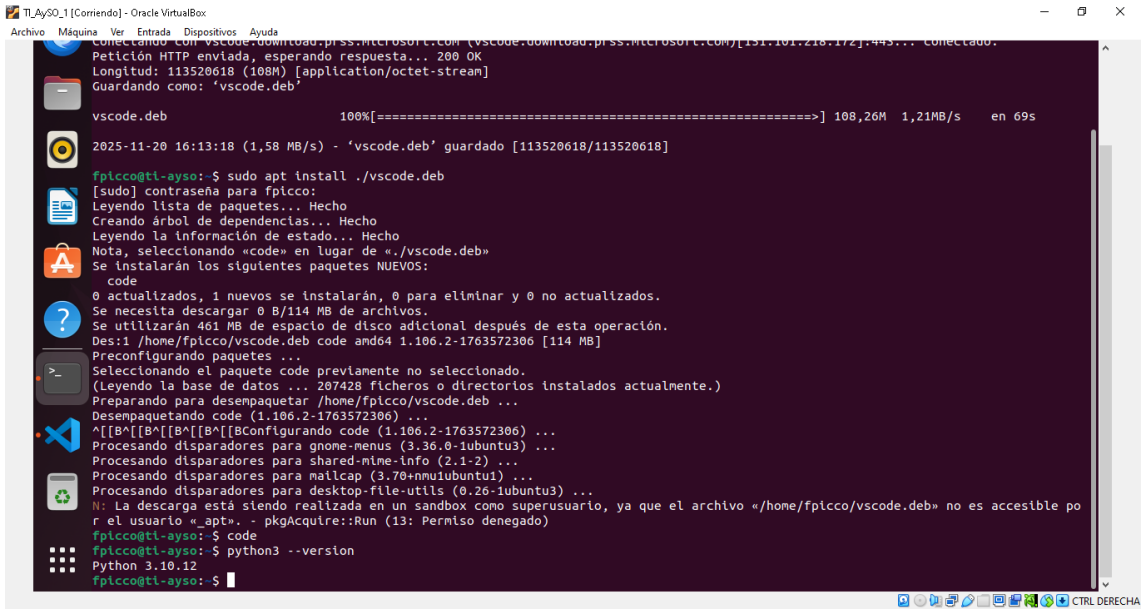
#### Anexo 4 – Instalación de VS Code

Procedí a utilizar la consola (ctrl+Alt+t) para instalar la aplicación **“Visual Studio Code”**. Dentro de Linux pude escribir código del programa para el caso práctico elegido.

## Arquitectura y Sistemas Operativos



- Verifiqué que **Python** esté instalado.



## Anexo 5 – Configuración de Git

- Instalación de **Git** y verificación

## TRABAJO PRÁCTICO INTEGRADOR

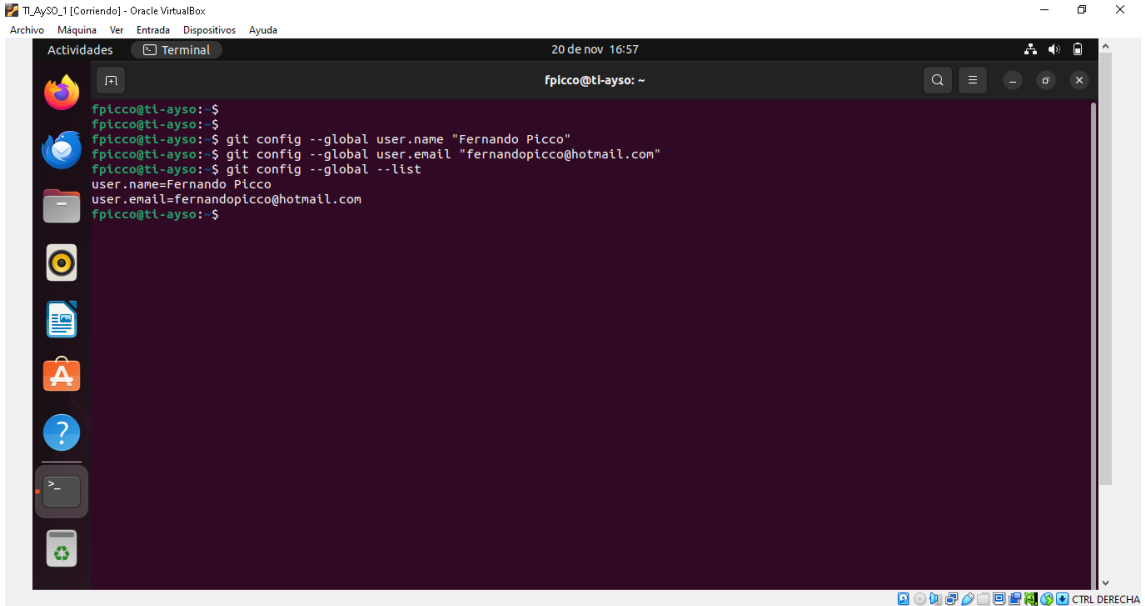
### Arquitectura y Sistemas Operativos



```
TL_AySO_1 [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Se está actualizando el estado... Hecho
Se pueden actualizar 5 paquetes. Ejecute «apt list --upgradable» para verlos.
fPICCO@tl-ayso:~$ sudo apt install git
Se está leyendo la lista de paquetes... Hecho
Se está leyendo el árbol de dependencias... Hecho
Se está leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
git-man liberror-perl
Paquetes sugeridos:
git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
Se instalarán los siguientes paquetes NUEVOS:
git git-man liberror-perl
Se actualizarán 3 paquetes y 0 nuevos se instalarán, 0 para eliminar y 5 no actualizados.
Se necesitan descargar 4.147 kB de archivos.
Se necesitarán 21,0 MB de espacio de disco adicional después de esta operación.
¿continuar? [S/n] S
Destino http://ar.archive.ubuntu.com/ubuntu jammy/main amd64 liberror-perl all 0.17029-1 [26,5 kB]
Destino http://ar.archive.ubuntu.com/ubuntu jammy-updates/main amd64 git-man all 1:2.34.1-1ubuntu1.15 [955 kB]
Destino http://ar.archive.ubuntu.com/ubuntu jammy-updates/main amd64 git amd64 1:2.34.1-1ubuntu1.15 [3.166 kB]
Se descargarán 4.147 kB en 5s (902 kB/s)
Se seleccionando el paquete liberror-perl previamente no seleccionado.
Se leyendo la base de datos ... 210614 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../liberror-perl_0.17029-1_all.deb ...
Desempaquetando liberror-perl (0.17029-1) ...
Se seleccionando el paquete git-man previamente no seleccionado.
Preparando para desempaquetar .../git-man_1:2.34.1-1ubuntu1.15_all.deb ...
Desempaquetando git-man (1:2.34.1-1ubuntu1.15) ...
Se seleccionando el paquete git previamente no seleccionado.
Preparando para desempaquetar .../git_1:2.34.1-1ubuntu1.15_amd64.deb ...
Desempaquetando git (1:2.34.1-1ubuntu1.15) ...
Configurando liberror-perl (0.17029-1) ...
Configurando git-man (1:2.34.1-1ubuntu1.15) ...
Configurando git (1:2.34.1-1ubuntu1.15) ...
Procesando disparadores para man-db (2.10.2-1) ...
fPICCO@tl-ayso:~$ git --version
git version 2.34.1
fPICCO@tl-ayso:~$
```

- Configuración de GitHub (mi usuario – nombre y mail - )



```
TL_AySO_1 [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

20 de nov 16:57
fPICCO@tl-ayso: ~

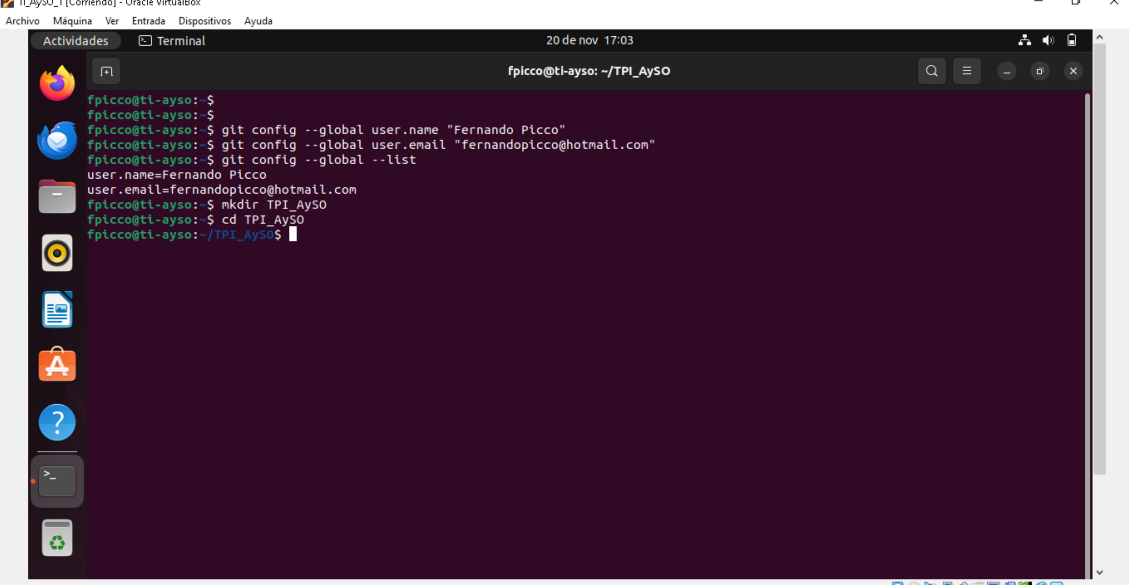
fPICCO@tl-ayso:~$
fPICCO@tl-ayso:~$
fPICCO@tl-ayso:~$ git config --global user.name "Fernando Picco"
fPICCO@tl-ayso:~$ git config --global user.email "fernandopicco@hotmail.com"
fPICCO@tl-ayso:~$ git config --global --list
user.name=Fernando Picco
user.email=fernandopicco@hotmail.com
fPICCO@tl-ayso:~$
```

- Creación de la carpeta del Trabajo Integrador dentro de Ubuntu ("TPI\_AySO")



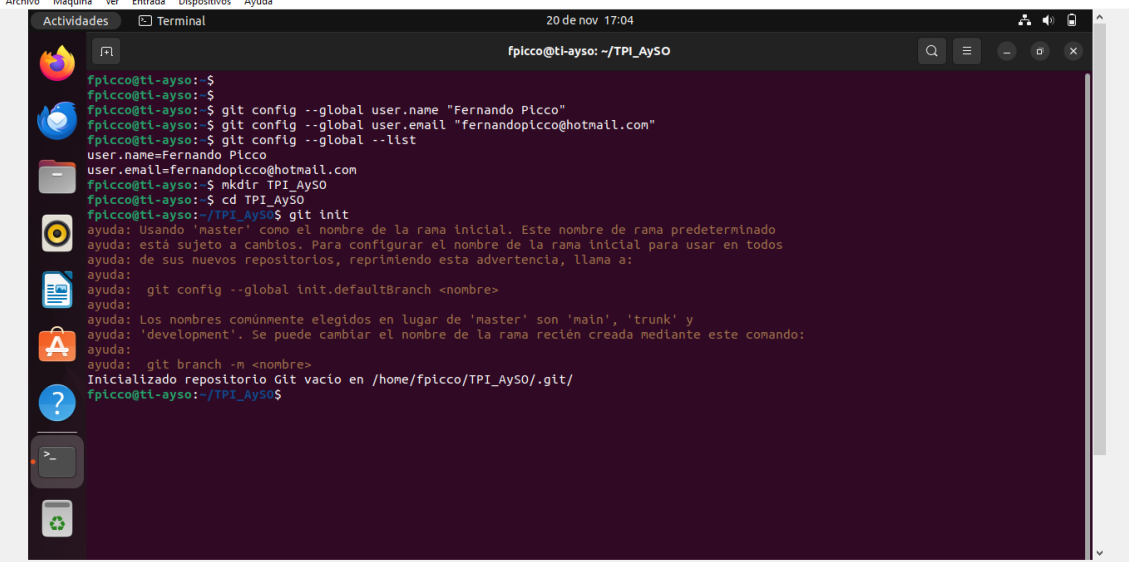
## TRABAJO PRÁCTICO INTEGRADOR

### Arquitectura y Sistemas Operativos



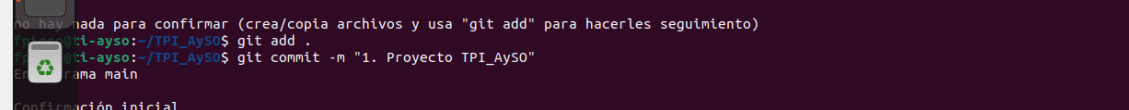
```
fpicco@ti-ayso: ~/TPI_AySO
fpicco@ti-ayso: $
fpicco@ti-ayso: $
fpicco@ti-ayso: $ git config --global user.name "Fernando Picco"
fpicco@ti-ayso: $ git config --global user.email "fernandopicco@hotmail.com"
fpicco@ti-ayso: $ git config --global --list
user.name=Fernando Picco
user.email=fernandopicco@hotmail.com
fpicco@ti-ayso: $ mkdir TPI_AySO
fpicco@ti-ayso: $ cd TPI_AySO
fpicco@ti-ayso: ~/TPI_AySO$
```

- Ingresé a la carpeta “TPI\_AySO” e inicialicé repositorio



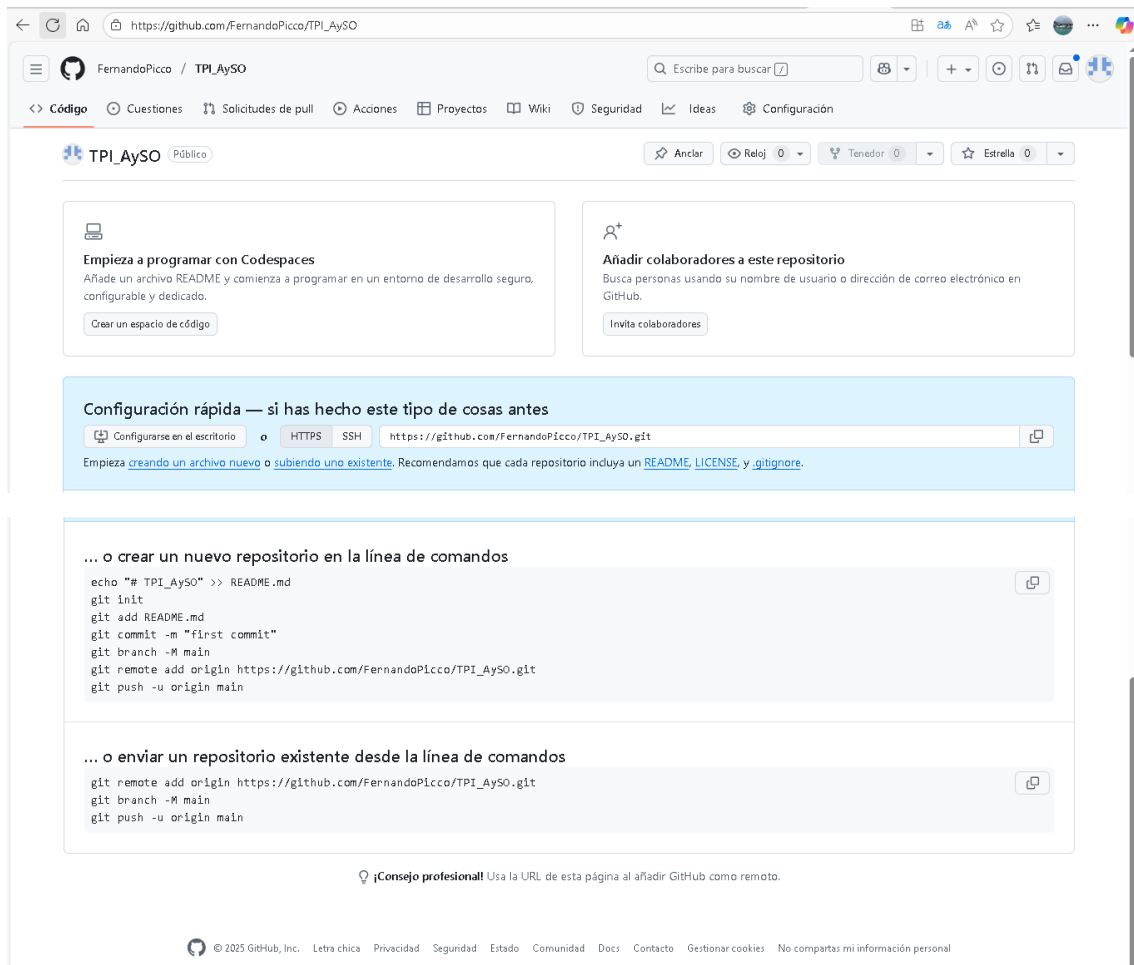
```
fpicco@ti-ayso: ~/TPI_AySO
fpicco@ti-ayso: $
fpicco@ti-ayso: $ git config --global user.name "Fernando Picco"
fpicco@ti-ayso: $ git config --global user.email "fernandopicco@hotmail.com"
fpicco@ti-ayso: $ git config --global --list
user.name=Fernando Picco
user.email=fernandopicco@hotmail.com
fpicco@ti-ayso: $ mkdir TPI_AySO
fpicco@ti-ayso: $ cd TPI_AySO
fpicco@ti-ayso: ~/TPI_AySO$ git init
ayuda: Usando 'master' como el nombre de la rama inicial. Este nombre de rama predeterminado
ayuda: está sujeto a cambios. Para configurar el nombre de la rama inicial para usar en todos
ayuda: de sus nuevos repositorios, reprimiendo esta advertencia, llama a:
ayuda:
ayuda: git config --global init.defaultBranch <nombre>
ayuda:
ayuda: Los nombres comúnmente elegidos en lugar de 'master' son 'main', 'trunk' y
ayuda: 'development'. Se puede cambiar el nombre de la rama recién creada mediante este comando:
ayuda:
ayuda: git branch -m <nombre>
ayuda:
ayuda: Inicializado repositorio Git vacío en /home/fpicco/TPI_AySO/.git/
fpicco@ti-ayso: ~/TPI_AySO$
```

- Agregué archivos y realicé el primer commit



```
fpicco@ti-ayso: ~/TPI_AySO$ git add .
fpicco@ti-ayso: ~/TPI_AySO$ git commit -m "1. Proyecto TPI_AySO"
ayuda: No hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles seguimiento)
ayuda:
ayuda: git commit -m <mensaje>
ayuda:
ayuda: Inicializado repositorio Git vacío en /home/fpicco/TPI_AySO/.git/
ayuda:
ayuda: git branch -m <nombre>
ayuda:
ayuda: Inicializado repositorio Git vacío en /home/fpicco/TPI_AySO/.git/
fpicco@ti-ayso: ~/TPI_AySO$
```

## Anexo 6 – Creación del repositorio GitHub (desde la web) y primeros commits



- Conexión del repositorio alojado en la web con la carpeta “TPI\_AySO” dentro de la máquina virtual

```
no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles seguimiento)
picco@ti-ayso:~/TPI_AySO$ git branch
picco@ti-ayso:~/TPI_AySO$ git remote -v
origin https://github.com/FernandoPicco/TPI_AySO.git (fetch)
origin https://github.com/FernandoPicco/TPI_AySO.git (push)
picco@ti-ayso:~/TPI_AySO$
```

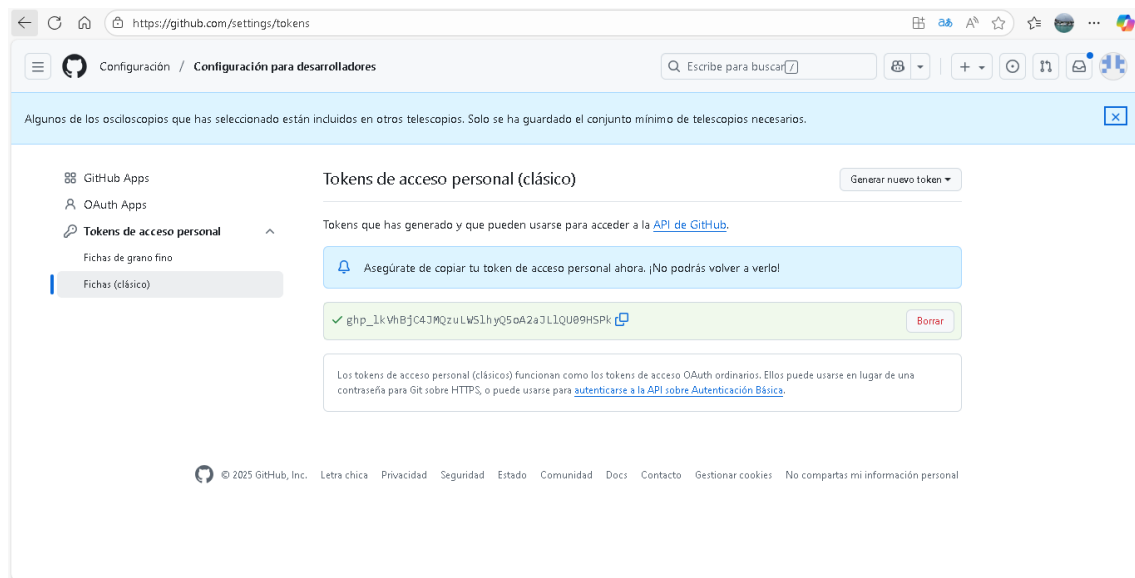
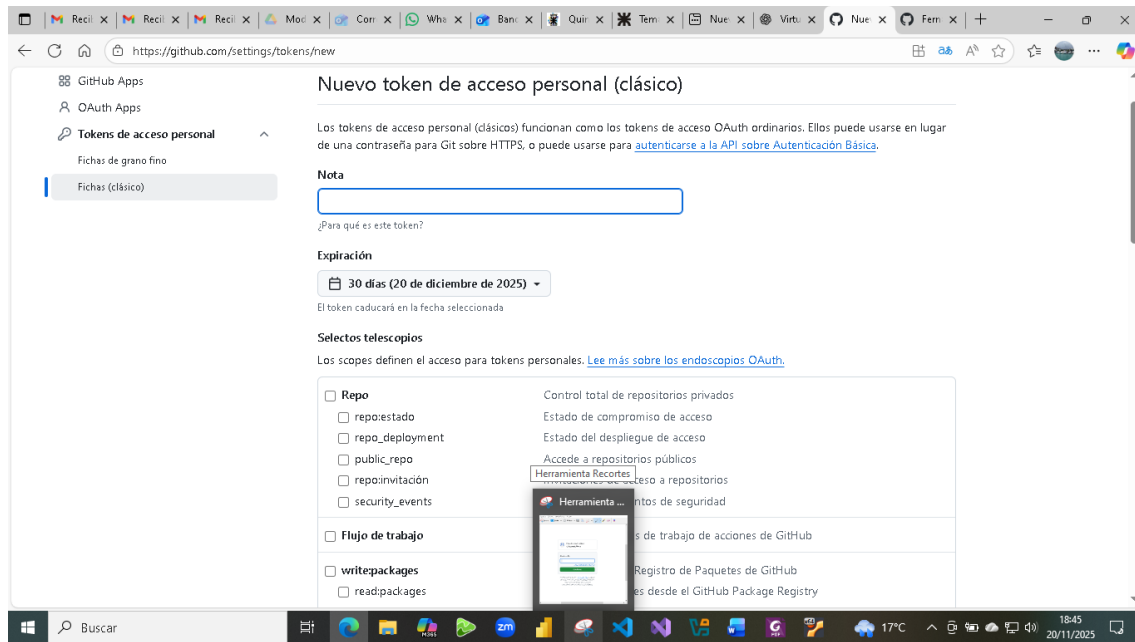
- Push final

```
? ti-ayso:~/TPI_AySO$ git status
On main
no commits todavía

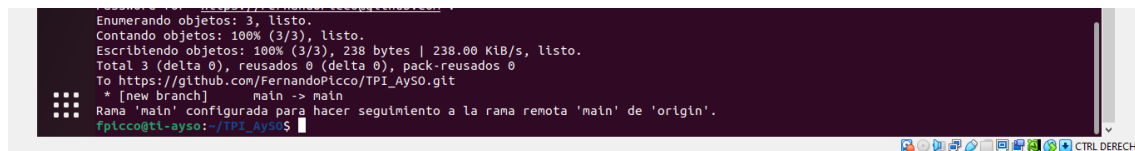
? ti-ayso:~/TPI_AySO$ git branch
ti-ayso:~/TPI_AySO$ echo "Proyecto TPI_AySO" > README.md
ti-ayso:~/TPI_AySO$ git add .
ti-ayso:~/TPI_AySO$ git commit -m "Commit inicial"
[main (commit-raiz) be656e6] Commit inicial
1 file changed, 1 insertion(+)
create mode 100644 README.md
picco@ti-ayso:~/TPI_AySO$ git branch
ti-ayso:~/TPI_AySO$ git branch
ti-ayso:~/TPI_AySO$ git push -u origin main
Username for 'https://github.com':
```

# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos



- Push realizado correctamente / repositorio sincronizado

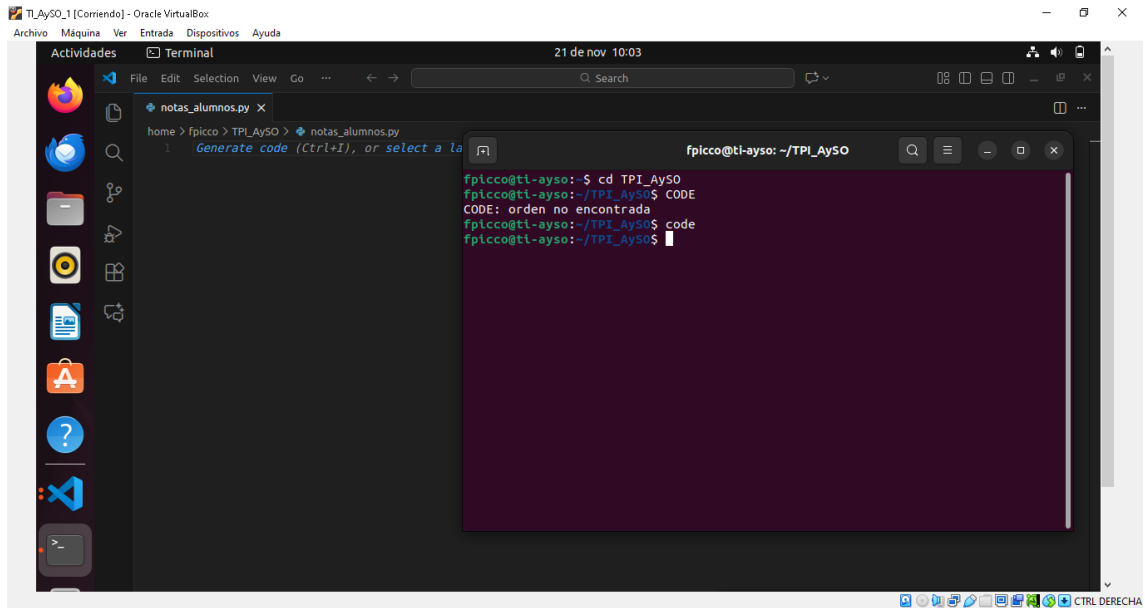


### Anexo 7 – desarrollo del programa en la Máquina Virtual

- 1) Preparación del archivo:
  - ✓ Ingreso a la carpeta del TP
  - ✓ Abrí VSC
  - ✓ Nuevo archivo para desarrollar el programa “notas\_alumnos.py”

# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos



- Corrección de error en el código

## TRABAJO PRÁCTICO INTEGRADOR

### Arquitectura y Sistemas Operativos

The screenshot shows a terminal window with a Python script named `notas_alumnos.py`. The script prompts for a student's name and three grades, calculates the average, and prints the student's name, average, and approval status. The terminal output shows the script being executed, but it encounters a `AttributeError: 'float' object has no attribute 'strip'` on line 7. The error message indicates that the `strip()` method is being called on a float object, which is incorrect.

```
1 # Programa de notas - Prueba 1: Calcula
2
3 print("****Cálculo de promedio de un alumno****")
4
5 nombre = input("Ingrese el nombre del alumno: ")
6 nota_1 = float(input("Ingrese la nota 1: "))
7 nota_2 = float(input("Ingrese la nota 2: "))
8 nota_3 = float(input("Ingrese la nota 3: "))
9
10 promedio = (nota_1 + nota_2 + nota_3) / 3
11
12 print(f"Alumno: {nombre}")
13 print(f"Promedio: {promedio:.2f}")
14
15 if promedio >= 7:
16     print("Estado: Aprobado")
17 else:
18     print("Estado: Desaprobado")
19
20
```

Terminal output:

```
fpicco@ti-ayso: ~/TPI_AySO
fpicco@ti-ayso:~/TPI_AySO$ cd TPI_AySO
fpicco@ti-ayso:~/TPI_AySO$ CODE
CODE: orden no encontrada
fpicco@ti-ayso:~/TPI_AySO$ code
fpicco@ti-ayso:~/TPI_AySO$ code
fpicco@ti-ayso:~/TPI_AySO$ python3 notas_alumnos.py
****Cálculo de promedio de un alumno****
Ingrese el nombre del alumno: Fernando Picco
Ingrese la nota 1: 5
Traceback (most recent call last):
  File "/home/fpicco/TPI_AySO/notas_alumnos.py", line 7, in <module>
    nota_1 = float(input("Ingrese la nota 1: ").strip())
AttributeError: 'float' object has no attribute 'strip'
fpicco@ti-ayso:~/TPI_AySO$
```

- Error solucionado

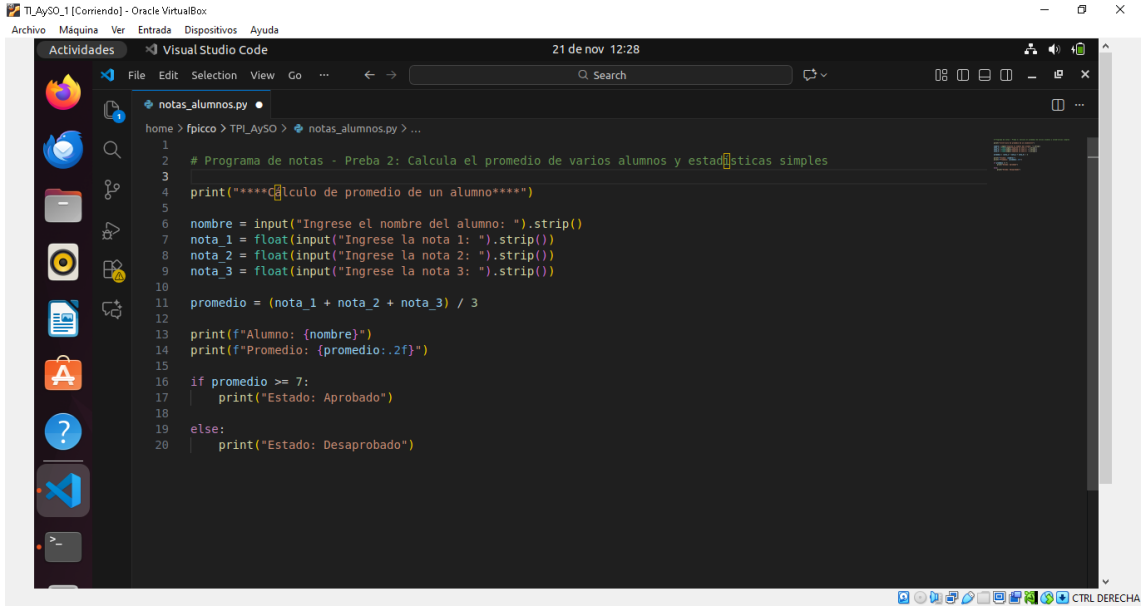
The screenshot shows the same terminal window, but the error has been resolved. The script is now running correctly, and the output shows the student's name, average, and approval status. The terminal output shows the script being executed, and the output is as follows:

```
Ingrese la nota 1: 5
Ingrese la nota 2: 5
Ingrese la nota 3: 5
Alumno: Fer Picco
Promedio: 11.67
Estado: Aprobado
fpicco@ti-ayso:~/TPI_AySO$ python3 notas_alumnos.py
****Cálculo de promedio de un alumno****
Ingrese el nombre del alumno: Fer
Ingrese la nota 1: 5
Ingrese la nota 2: 5
Ingrese la nota 3: 5
Alumno: Fer
Promedio: 5.00
Estado: Desaprobado
fpicco@ti-ayso:~/TPI_AySO$
```

- Commit N°1: Prueba 1: promedio de un alumno y estado de aprobación

# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos



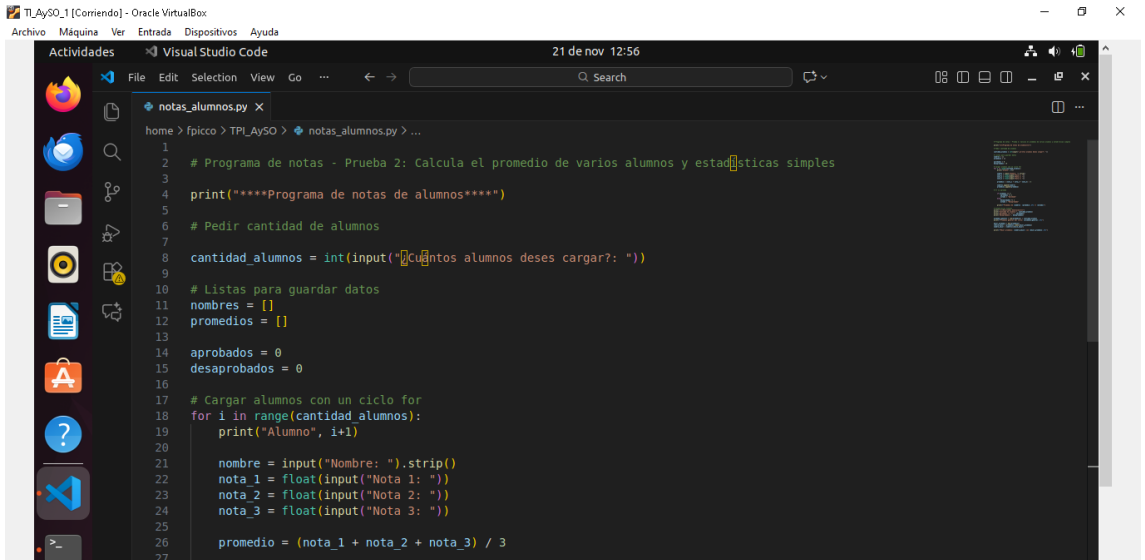
The screenshot shows the Visual Studio Code editor with a file named `notas_alumnos.py` open. The code is a Python script for calculating the average of three student grades and determining their status. The code is as follows:

```
1 # Programa de notas - Preba 2: Calcula el promedio de varios alumnos y estadísticas simples
2
3 print("*****Cálculo de promedio de un alumno*****")
4
5 nombre = input("Ingrese el nombre del alumno: ").strip()
6 nota_1 = float(input("Ingrese la nota 1: ").strip())
7 nota_2 = float(input("Ingrese la nota 2: ").strip())
8 nota_3 = float(input("Ingrese la nota 3: ").strip())
9
10 promedio = (nota_1 + nota_2 + nota_3) / 3
11
12 print(f"Alumno: {nombre}")
13 print(f"Promedio: {promedio:.2f}")
14
15 if promedio >= 7:
16     print("Estado: Aprobado")
17 else:
18     print("Estado: Desaprobado")
19
20
```

```
fpicco@ti-ayso:~/TPI_AySO$ git add .
fpicco@ti-ayso:~/TPI_AySO$ git commit -m "Prueba 1: prom-est-alumn"
En la rama main
Tu rama está actualizada con 'origin/main'.

nada para hacer commit, el árbol de trabajo está limpio
fpicco@ti-ayso:~/TPI_AySO$ git push
Everything up-to-date
fpicco@ti-ayso:~/TPI_AySO$
```

- Commit N°2: Prueba 2: promedio varios alumnos y estadísticas simples



The screenshot shows the Visual Studio Code editor with a file named `notas_alumnos.py` open. The code is a Python script for calculating the average of three student grades and determining their status, but it includes a loop to handle multiple students. The code is as follows:

```
1 # Programa de notas - Prueba 2: Calcula el promedio de varios alumnos y estadísticas simples
2
3 print("*****Programa de notas de alumnos*****")
4
5 # Pedir cantidad de alumnos
6
7 cantidad_alumnos = int(input("¿Cuántos alumnos deses cargar?: "))
8
9 # Listas para guardar datos
10 nombres = []
11 promedios = []
12
13 aprobados = 0
14 desaprobados = 0
15
16 # Cargar alumnos con un ciclo for
17 for i in range(cantidad_alumnos):
18     print("Alumno", i+1)
19
20     nombre = input("Nombre: ").strip()
21     nota_1 = float(input("Nota 1: "))
22     nota_2 = float(input("Nota 2: "))
23     nota_3 = float(input("Nota 3: "))
24
25     promedio = (nota_1 + nota_2 + nota_3) / 3
26
27
```

# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos

```
27
28     nombres.append(nombre)
29     promedios.append(promedio)
30
31 # Vr si aprueba
32
33 if promedio >= 7:
34     aprobados += 1
35     estado = "Aprobado"
36 else:
37     desaprobados += 1
38     estado = "Desaprobado"
39
40 print(f"Promedio de {nombre}: {promedio:.2f} => {estado}")
41
42 # Estadísticas finales
43 print("****Resultados Generales****")
44 print("Cantidad de alumnos:", cantidad_alumnos)
45 print("Aprobados (>= 7):", aprobados)
46 print("Desaprobados:", desaprobados)
47
48 promedio_general = sum(promedios) / cantidad_alumnos
49 print(f"Promedio general del curso: {promedio_general:.2f}")
50
51 mejor_promedio = max(promedios)
52 indice_mejor = promedios.index(mejor_promedio)
53 nombre_mejor = nombres[indice_mejor]
54
55 print(f"Mejor promedio: {nombre_mejor} con {mejor_promedio:.2f}")
56
57
58
59
```

```
TLAy50_1 [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Actividades  Terminal  21 de nov 13:00

fpicco@tl-ayso: ~/TPI_AySO

notas_alumnos.py X
home > fpicco > TPI_AySO > notas_alu ****Programa de notas de alumnos****
1 Cuántos alumnos deses cargar?: 3
2 Alumno 1
3 # Programa de notas - Pr Nombre: Fer
4 Nota 1: 5
5 print("****Programa de h Nota 2: 5
6 Nota 3: 5
7 Promedio de Fer: 5.00 => Desaprobado
8 Alumno 2
9 Nombre: Ana
10 Nota 1: 4
11 Nota 2: 4
12 Nota 3: 4
13 Promedio de Ana: 4.00 => Desaprobado
14 Alumno 3
15 Nombre: Carlos
16 Nota 1: 6
17 Nota 2: 6
18 Nota 3: 6
19 Promedio de Carlos: 6.00 => Desaprobado
20 ****Resultados Generales****
21 Cantidad de alumnos: 3
22 Aprobados (>= 7): 0
23 Desaprobados: 3
24 Promedio general del curso: 5.00
25 Mejor promedio: Carlos con 6.00
26 fpicco@tl-ayso: ~/TPI_AySO$ git status
27 En la rama main
28 Tu rama está actualizada con 'origin/main'.
29
30 Cambios no rastreados para el commit:
31 (usa "git add <archivo>..." para actualizar lo que será confirmado)
32 (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
33 modificados: notas_alumnos.py
34
35 sin cambios agregados al commit (usa "git add" y/o "git commit -a")
36 fpicco@tl-ayso: ~/TPI_AySO$ git add notas_alumnos.py
37 fpicco@tl-ayso: ~/TPI_AySO$ git commit -m "Prueba 2: carga + alumn + estad grales"
38 [main f27e20c] Prueba 2: carga + alumn + estad grales
39 1 file changed, 59 insertions(+), 20 deletions(-)
40 rewrite notas_alumnos.py (98%)
41 fpicco@tl-ayso: ~/TPI_AySO$ git push
42 Enumerando objetos: 5, listo.
43 Contando objetos: 100% (5/5), listo.
44 Compresión delta usando hasta 2 hilos
45 Comprimiendo objetos: 100% (3/3), listo.
46 Escribiendo objetos: 100% (3/3), 889 bytes | 889.00 KiB/s, listo.
47 Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
48 To https://github.com/FernandoPicco/TPI_AySO.git
49 4d1fba4..f27e20c main -> main
50 fpicco@tl-ayso: ~/TPI_AySO$
```

```
Promedio de Carlos: 6.00 => Desaprobado
****Resultados Generales****
Cantidad de alumnos: 3
Aprobados (>= 7): 0
Desaprobados: 3
Promedio general del curso: 5.00
Mejor promedio: Carlos con 6.00
fpicco@tl-ayso: ~/TPI_AySO$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

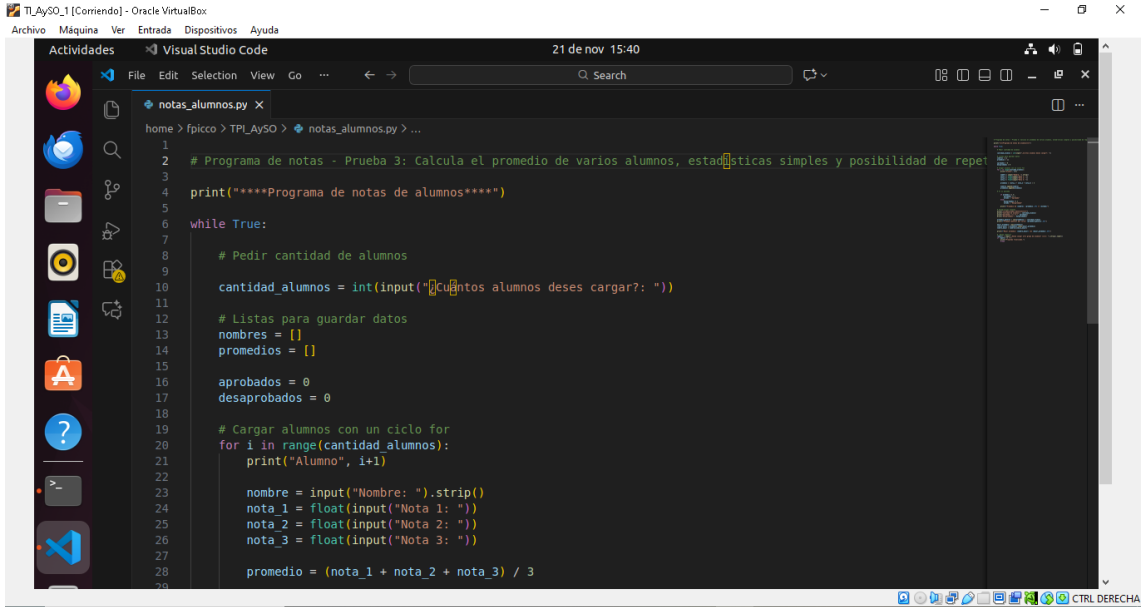
Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
modificados: notas_alumnos.py

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
fpicco@tl-ayso: ~/TPI_AySO$ git add notas_alumnos.py
fpicco@tl-ayso: ~/TPI_AySO$ git commit -m "Prueba 2: carga + alumn + estad grales"
[main f27e20c] Prueba 2: carga + alumn + estad grales
1 file changed, 59 insertions(+), 20 deletions(-)
rewrite notas_alumnos.py (98%)
fpicco@tl-ayso: ~/TPI_AySO$ git push
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 889 bytes | 889.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://github.com/FernandoPicco/TPI_AySO.git
4d1fba4..f27e20c main -> main
fpicco@tl-ayso: ~/TPI_AySO$
```

- Commit N°3: Prueba 3: promedio varios alumnos y estadísticas simples y preguntar si desea repetir la carga.

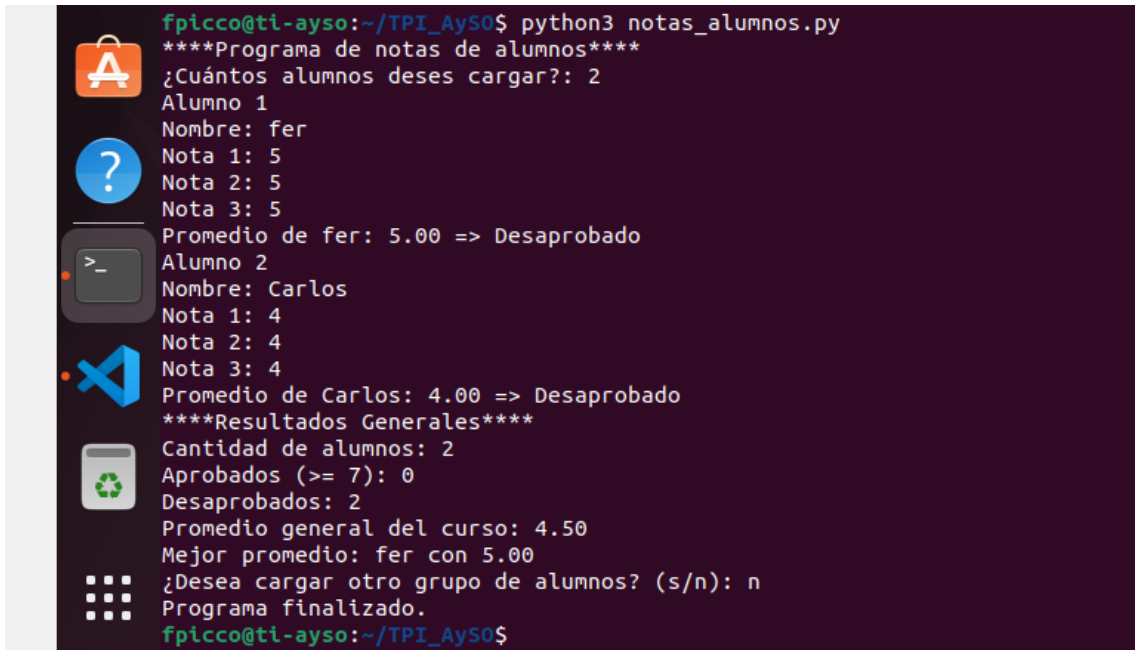
# TRABAJO PRÁCTICO INTEGRADOR

## Arquitectura y Sistemas Operativos

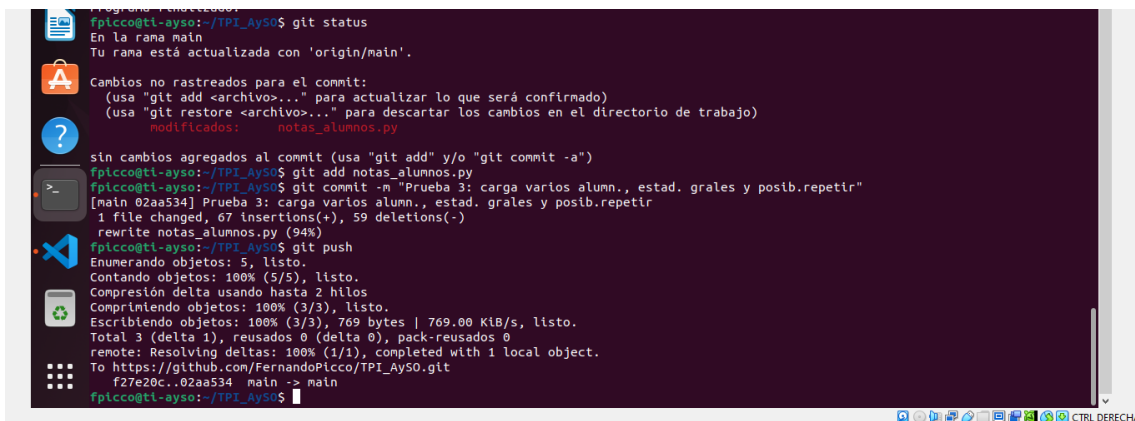


```
1 # Programa de notas - Prueba 3: Calcula el promedio de varios alumnos, estadísticas simples y posibilidad de repetir
2
3 print("****Programa de notas de alumnos****")
4
5 while True:
6     # Pedir cantidad de alumnos
7
8     cantidad_alumnos = int(input("¿Cuántos alumnos deses cargar?: "))
9
10    # Listas para guardar datos
11    nombres = []
12    promedios = []
13
14    aprobados = 0
15    desaprobados = 0
16
17    # Cargar alumnos con un ciclo for
18    for i in range(cantidad_alumnos):
19        print("Alumno", i+1)
20
21        nombre = input("Nombre: ").strip()
22        nota_1 = float(input("Nota 1: "))
23        nota_2 = float(input("Nota 2: "))
24        nota_3 = float(input("Nota 3: "))
25
26        promedio = (nota_1 + nota_2 + nota_3) / 3
```

- Se prueba que funciona bien:



```
fpicco@ti-ayso:~/TPI_AySO$ python3 notas_alumnos.py
****Programa de notas de alumnos****
¿Cuántos alumnos deses cargar?: 2
Alumno 1
Nombre: fer
Nota 1: 5
Nota 2: 5
Nota 3: 5
Promedio de fer: 5.00 => Desaprobado
Alumno 2
Nombre: Carlos
Nota 1: 4
Nota 2: 4
Nota 3: 4
Promedio de Carlos: 4.00 => Desaprobado
****Resultados Generales****
Cantidad de alumnos: 2
Aprobados (>= 7): 0
Desaprobados: 2
Promedio general del curso: 4.50
Mejor promedio: fer con 5.00
¿Desea cargar otro grupo de alumnos? (s/n): n
Programa finalizado.
fpicco@ti-ayso:~/TPI_AySO$
```



```
fpicco@ti-ayso:~/TPI_AySO$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
modificados:   notas_alumnos.py

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
fpicco@ti-ayso:~/TPI_AySO$ git add notas_alumnos.py
fpicco@ti-ayso:~/TPI_AySO$ git commit -m "Prueba 3: carga varios alumn., estad. gales y posib.repetitr"
[main 02aa534] Prueba 3: carga varios alumn., estad. gales y posib.repetitr
1 file changed, 67 insertions(+), 59 deletions(-)
rewrite notas_alumnos.py (94%)
fpicco@ti-ayso:~/TPI_AySO$ git push
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 769 bytes | 769.00 KiB/s, listo.
Total 3 (delta 1), reusados 0 (delta 0), pack-reusados 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/FernandoPicco/TPI_AySO.git
f27e20c..02aa534  main -> main
fpicco@ti-ayso:~/TPI_AySO$
```